



## OPEN EcoTaskSched: a hybrid machine learning approach for energy-efficient task scheduling in IoT-based fog-cloud environments

Asfandiyar Khan<sup>1</sup>, Faizan Ullah<sup>2</sup>, Dilawar Shah<sup>2✉</sup>, Muhammad Haris Khan<sup>2</sup>, Shujaat Ali<sup>2</sup> & Muhammad Tahir<sup>3✉</sup>

The widespread adoption of cloud services has posed several challenges, primarily revolving around energy and resource efficiency. Integrating cloud and fog resources can help address these challenges by improving fog-cloud computing environments. Nevertheless, the search for optimal task allocation and energy management in such environments continues. Existing studies have introduced notable solutions; however, it is still a challenging issue to efficiently utilize these heterogeneous cloud resources and achieve energy-efficient task scheduling in fog-cloud of things environment. To tackle these challenges, we propose a novel ML-based EcoTaskSched model, which leverages deep learning for energy-efficient task scheduling in fog-cloud networks. The proposed hybrid model integrates Convolutional Neural Networks (CNNs) with Bidirectional Log-Short Term Memory (BiLSTM) to enhance energy-efficient schedulability and reduce energy usage while ensuring QoS provisioning. The CNN model efficiently extracts workload features from tasks and resources, while the BiLSTM captures complex sequential information, predicting optimal task placement sequences. A real fog-cloud environment is implemented using the COSCO framework for the simulation setup together with four physical nodes from the Azure B2s plan to test the proposed model. The DeFog benchmark is used to develop task workloads, and data collection was conducted for both normal and intense workload scenarios. Before preprocessing the data was normalized, treated with feature engineering and augmentation, and then split into training and test sets. To evaluate performance, the proposed EcoTaskSched model demonstrated superiority by significantly reducing energy consumption and improving job completion rates compared to baseline models. Additionally, the EcoTaskSched model maintained a high job completion rate of 85%, outperforming GGCCN and BiGGCCN. It also achieved a lower average response time, and SLA violation rates, as well as increased throughput, and reduced execution cost compared to other baseline models. In its optimal configuration, the EcoTaskSched model is successfully applied to fog-cloud computing environments, increasing task handling efficiency and reducing energy consumption while maintaining the required QoS parameters. Our future studies will focus on long-term testing of the EcoTaskSched model in real-world IoT environments. We will also assess its applicability by integrating other ML models, which could provide enhanced insights for optimizing scheduling algorithms across diverse fog-cloud settings.

**Keywords** AI, Cloud, Convolutional neural networks, Fog, IoT, ML, SLA, Task scheduling, QoS

### Abbreviations

|         |                                                 |
|---------|-------------------------------------------------|
| AI      | Artificial intelligence                         |
| AHB     | Advanced high-performance bus                   |
| ANN     | Artificial neural networks                      |
| BiGGCCN | Bidirectional gated graph convolutional network |
| BiGRUs  | Bidirectional gated recurrent units             |

<sup>1</sup>Department of Computer Science and Information Technology, Hazara University Mansehra, Dhodial 21120, Pakistan. <sup>2</sup>Department of Computer Science, Bacha Khan University, Charsadda 24420, Pakistan.

<sup>3</sup>Department of Computer Science, Kardan University, Kabul, Afghanistan. ✉email: dilawar\_shah@bkuc.edu.pk; m.tahir@kardan.edu.af

|                 |                                                              |
|-----------------|--------------------------------------------------------------|
| BiLSTM          | Bidirectional long-short term memory                         |
| COSCO           | Couple-simulation container orchestration                    |
| CNN             | Convolutional neural networks                                |
| CPU             | Central processing unit                                      |
| 2D              | Two dimensional                                              |
| EVs             | Electric vehicles                                            |
| Eqs             | Equations                                                    |
| FCFS            | First-come first-served                                      |
| GA              | Genetic algorithm                                            |
| GB/GiB          | Giga byte/gibibyte                                           |
| GPU             | Graphical processing unit                                    |
| H               | Host                                                         |
| I               | Interval                                                     |
| IaaS            | Infrastructure as a service                                  |
| ICT             | Information communication and technology                     |
| IO              | Input output operations (in MB/s)                            |
| IoT             | Internet of Things                                           |
| IT              | Information technology                                       |
| kWh             | Kilo watt per hour                                           |
| MB              | Megabyte                                                     |
| MIPS            | Million instructions per second                              |
| ML              | Machine learning                                             |
| ms              | Milliseconds                                                 |
| NP-PQS          | Non preemptive-priority queue scheduling                     |
| OS              | Operating system                                             |
| PaaS            | Platform as a service                                        |
| PP              | Preemptive priority                                          |
| $p_y, p_a, p_s$ | Probabilities of Yolo, Aeneas, and PocketSphinx applications |
| RAM             | Random access memory                                         |
| RL              | Reinforcement learning                                       |
| RNN             | Recurrent neural network                                     |
| RR              | Round robin                                                  |
| SaaS            | Software as a service                                        |
| SLA             | Service level agreement                                      |
| TaaS            | Task as a service                                            |
| QoS             | Quality of service                                           |
| UK/US           | United Kingdom/United States                                 |
| VMs             | Virtual machines                                             |
| Wi-Fi           | Wireless fidelity                                            |

In recent years, cloud computing has revolutionized how business organizations manage and deploy their ICT<sup>1</sup> resources by offering scalable, on-demand services<sup>2</sup> to service users. This paradigm shift enables businesses to reduce infrastructure costs related to hardware procurement, maintenance, and energy consumption by transferring these responsibilities to cloud service providers<sup>3</sup>. However, the exponential growth in cloud service usage including application delivery, storage, computation, and resource allocation has led to significant challenges, particularly in energy efficiency and resource management. Fog computing<sup>4</sup>, an extension of cloud computing<sup>5</sup>, aims to address some of these challenges<sup>6</sup> by decentralizing data processing closer to the data source<sup>7,8</sup>. This approach reduces latency and bandwidth usage, which are critical for real-time applications such as the Internet of Things (IoT)<sup>9</sup>, autonomous vehicles<sup>10</sup>, smart grids<sup>11</sup>, smart farming<sup>12</sup>, smart cities<sup>13</sup>, industries<sup>14,15</sup> and healthcare<sup>16,17</sup>. By processing data at the edge of the network, fog computing minimizes the need for data to travel long distances to centralized cloud servers, thereby enhancing performance and reliability. Fog-cloud computing environments inherit the strengths of both cloud and fog computing. The hybrid model leverages the centralized processing power of the cloud while utilizing the proximity and low latency of fog nodes<sup>18</sup>. Additionally, it supports a wide range of applications, from consumer-oriented services to critical industrial operations. Such an environment can efficiently handle the vast amounts of data generated by IoT devices, enabling real-time analytics and decision-making more convenient<sup>19</sup>. Another key benefit of fog-cloud computing is its ability to provide a scalable and flexible infrastructure. Organizations can dynamically allocate resources in a cost-effective manner<sup>20</sup> to achieve optimal system performance.

Despite these prominent features, fog-cloud computing faces several design challenges, particularly in the context of task scheduling and energy efficiency<sup>21,22</sup>. Energy-efficient task scheduling is crucial to ensuring that computational tasks are assigned to the most appropriate resources, whether in the cloud or at the fog nodes. This requires sophisticated algorithms capable of balancing the load, minimizing energy consumption, and maintaining high levels of performance<sup>23</sup>. Recent advancements in artificial intelligence (AI) and machine learning (ML) have shown promise in addressing these challenges<sup>24</sup>. ML-based task scheduling algorithms optimize resource allocation by predicting workloads and dynamically adjusting task assignments and execution. Similarly, the growing demand for cloud services has led to increased power consumption in data centers. Intelligent systems can significantly reduce energy consumption and enhance the overall performance of the fog-cloud environment. In addition, data centers that host heterogeneous servers, storage, and network devices incur higher operating costs due to their capacity requirements. As these computing environments scale up and

consume large amount of electricity to run host servers, support communication networks, and power cooling systems, mitigating power waste and reducing operating costs in large IT projects becomes a major concern.

In contrast, many researchers have adopted various ML techniques, carefully considering sampling data and feature sets to train and test scheduling policies in the context of fog-cloud computing environments. However, their published works contain shortcomings related to resource allocation, optimization, task scheduling, and power management in fog-cloud computing.

These challenges motivate us to clearly address this research challenge and provide a viable solution to the existing work in this domain. Therefore, this research study mainly focuses on developing an energy-efficient task scheduling framework, ensuring that each task is strictly assigned to either a fog or cloud node. This study aims to ensure that the response time does not exceed a maximum threshold, a minimum job completion rate is maintained under intense workloads, and SLA violations are limited to a fixed threshold in fog-cloud environments.

Inspired by the baseline models<sup>25,26</sup>, this research study extends our previously published work<sup>27</sup> by proposing a novel ML-based hybrid scheduling approach called EcoTaskSched model. The goal of EcoTaskSched model is to improve workflow efficiency by scheduling energy-efficient tasks in fog and cloud resources. Additionally, it aims to reduce the energy footprint by leveraging advanced deep learning (DL) techniques. This is achieved through the concatenation of a CNN model with a BiLSTM network. When compared to existing research, the proposed EcoTaskSched model provides a viable solution. It ensures that all performance metrics required for various dynamic IoT applications are met for the first time in a fog-cloud environment. Our significant contributions in this research study include the following:

1. The EcoTaskSched model is introduced as a novel task scheduling architecture integrating: (i) A CNN, which typically has multiple deep layers designed to detect and extract spatial patterns (feature sets) from tasks and system resources. (ii) A BiLSTM, combined with convolutional layers, to capture both future and past context from the input sequence to solve the research problem.
2. The research problem is mathematically formulated through an objective function, subject to additional operating constraints. An algorithm is implemented to apply the proposed methodology for energy-efficient task scheduling in fog and cloud resources.
3. Finally, we evaluate the performance of the proposed EcoTaskSched model through extensive experiments simulated using the Coupled Simulation and Container Orchestration (COSCO) framework under different workload scenarios. Simulation results show that the objective function is satisfied, indicating that energy consumption is minimized and QoS requirements are met for heterogeneous IoT applications in a fog-cloud computing environment.

The remainder of this article is organized as follows: Section "[Related work](#)" introduces related research, divided into subparts covering traditional task scheduling algorithms and ML-based scheduling techniques for fog-cloud computing tasks. Section "[Research problem](#)" defines the research problem and mathematically formulates it through the objective function. Section "[Overview of ML-based models and the system architecture](#)" provides an overview of ML-based models and the system architecture. Section "[Proposed EcoTaskSched methodology](#)" describes the research methodology of the proposed EcoTaskSched model in detail. Section "[Performance evaluation](#)" presents the experimental setup and evaluation metrics. In Section "[Results and discussion](#)", results are analyzed, visualized, and discussed, highlighting the significance of our proposed methodology. Finally, Conclusions are drawn and future directions are outlined in Section "[Conclusion and future work](#)".

## Related work

This section reviews key contemporary approaches adopted by researchers. The focus is on integrating traditional scheduling and ML-based scheduling techniques to achieve energy efficiency for distributed resources in a fog-cloud computing environment. The following subsections review related research relevant to our proposed research methodology.

### Traditional algorithms in task scheduling

Traditional task scheduling algorithms have been foundational in managing computing resources in cloud computing environments. These algorithms aim to balance and optimize resource allocation while reducing uptime and energy consumption in major cloud computing projects. Some commonly used traditional scheduling algorithms include First-Come, First-Served (FCFS), Round Robin (RR), Max-Min, Min-Min, Priority-Aware, and Most Efficient Server First (MESF).

An Energy-Aware Task Scheduler (ETSG)<sup>28</sup> uses a Genetic Algorithm (GA) to minimize both makespan and energy usage in cloud data centers. The proposed framework outperforms the Max-Min algorithm in reducing energy consumption and task execution time on VMs in cloud data centers. This is because the Max-Min algorithm assigns the task with the highest processing time to the server. This method prioritizes long tasks, helping to balance the load on servers. However, the Max-Min system can lead to energy inefficiency, as it does not minimize the system's total energy consumption. Similarly, a GA-based task scheduling approach<sup>29</sup> is proposed to uniformly distribute the load on VMs and minimize response time, which is a QoS requirement in a dynamic cloud environment. The proposed technique outperforms traditional scheduling methods such as FCFS and Greedy-based algorithms. Among these, FCFS is one of the simplest scheduling frameworks available. However, in FCFS, tasks are assigned inadvertently, without considering the specific needs or nature of tasks and resources in cloud paradigms. Although easy to implement, FCFS typically results in lower resource efficiency and higher energy consumption, as it does not account for server workloads or their ability to handle incoming tasks.

Many researchers have introduced modified versions of the basic Min-Min heuristic<sup>30</sup> to address task scheduling issues. To achieve efficient system performance, the Optimized Min-Min (OMin-Min) mechanism<sup>31</sup> is proposed. It classifies scientific workflows and scheduling strategies based on existing scheduling algorithms (i.e. Min-Min, Max-Min, Modified Max-Min, and Round Robin) for different cloudlet sizes to achieve better execution time and optimal resource utilization in a cloud computing environment. However, these mechanisms are not well-suited for heterogeneous resources in a multi-cloud task scheduling environment<sup>32</sup>, as they do not consider differences among cloudlets (tasks) and assume that all resources are homogeneous. In contrast, MESF<sup>33</sup> is a resource-aware scheduling framework that prioritizes tasks based on server efficiency. It identifies the most energy-efficient server for each task (job) and assigns tasks accordingly. Jobs are then allocated to the next most efficient server and continuing the process until all jobs are processed. This method can significantly reduce energy consumption while ensuring that response time constraints are met, compared to other random or less sophisticated scheduling methods. It is especially effective in heterogeneous environments where servers have varying capacities and energy profiles. An efficient task scheduling model using hybrid optimization techniques, incorporating Particle Swarm Grey Wolf Optimization (PSGWO), is proposed<sup>34</sup> to identify optimal VMs and allocate resources to tasks in cloud computing environments. For tasks scheduling, a support vector machine (SVM) classifier is used to categorize VMs statuses into four different categories. Linear parameters (for average utilization) and nonlinear parameters (for underutilization) are identified to assess VM utilization. To ensure QoS, VM status, waiting time and task execution time are considered to evaluate system performance and overcome tasks scheduling issues. QoS-aware resource optimization strategies for constrained IoT systems coupled with cloud computing environments are proposed in<sup>27,35</sup>. In<sup>27</sup>, a hybrid queue scheduling (HQS) method is introduced, using non-preemptive priority queue scheduling (NP-PQS) for time-critical IoT traffic and FCFS for normal traffic to guarantee QoS requirements for resource-constrained devices in the communication network. The proposed optimization method reduces the utilization cost of cloud resources (RAM, CPU, and bandwidth) for different IoT applications on cloud servers. However, the NP-PQS scheme in cloud computing faces several challenges, such as handling workloads, scalability issues, and resource starvation. A QoS-aware data aggregation (QoS-DA) strategy for different IoT applications (on-demand, periodic, and critical) with varying latency and traffic rates is proposed in<sup>35</sup> using a hierarchical clustering topology. The main objective of this strategy is to efficiently utilize the limited resources (RAM, CPU, network capacity, and storage) of relay devices in an IoT-enabled cloud computing network. Results show that the proposed strategy successfully optimized constrained resources, ensured QoS levels, and reduced execution time for IoT applications on cloud servers. However, such strategies come with several challenges, such as reduced data accuracy, vulnerable to data loss, and loss of data granularity. The research work in<sup>36</sup> proposes a job-oriented scheduling model for efficient resource scheduling in a cloud environment. Tasks are scheduled based on user preferences and available resources. Three traditional scheduling algorithms, namely Preemptive Priority (PP), RR, and Shortest Remaining Time First (SRTF) are used for resource scheduling. Among these, SRTF incurred the lowest time parameters and remained the most efficient scheduling algorithm. A Security Oriented Deadline-Aware (SODA) workflow allocation strategy is proposed in<sup>37</sup>, ensuring that each task meets security precedence demands and deadline constraints to minimize risk probability in an Infrastructure as a Service (IaaS) deterministic cloud computing environment. The scientific workflow order is calculated based on the security upward rank of each task and then allocated to and executed on trustable VMs that ensure the task's deadline and trust level. Simulation results show that SODA outperforms other strategies while reducing the risk probability of scientific workflows in the cloud systems. Table 1 provides a summary of recent research on resource scheduling problems in cloud computing.

Machine learning (ML)-based models in task scheduling

Nowadays, AI-enabled devices have introduced new capabilities to traditional task scheduling approaches, leading to smarter and more flexible design solutions in cloud computing environments. Recent research focuses on integrating ML-based models with traditional programming approaches to develop hybrid algorithms that better address task scheduling problems in today's complex fog-cloud computing environments. These advanced ML-based models learn from historical data, optimize resource allocation, and improve dynamic processing in real-time, significantly enhancing the ability to predict performance and optimize policy decisions for energy

| Refs.    | Technique      | Evaluation tool                         | Environment        | Properties                                                        |
|----------|----------------|-----------------------------------------|--------------------|-------------------------------------------------------------------|
| 27       | HQS            | CloudSim                                | IoT-cloud          | Reduce cost of cloud resource usage                               |
| 28       | ETSG           | CloudSim                                | Data centers-cloud | Reduce energy consumption, operational cost, and makespan         |
| 29       | DVS            | n/a                                     | Cloud              | Reduce response time (QoS)                                        |
| 30       | Min-Min        | Grid simulator                          | Cloud              | Ensure QoS                                                        |
| 31       | OMin-Min       | CloudSim                                | Multi-cloud        | Better execution time and optimize resources                      |
| 33       | MESF           | Matlab                                  | Data centers-cloud | Minimize energy consumption                                       |
| 34       | PSGWO          | NetBeans 8.1                            | Cloud              | Optimize VM and resource allocation                               |
| 35       | QoS-DA         | CloudSim                                | IoT-cloud          | Ensure QoS, reduce execution time and cost                        |
| 36       | PP, RR, & SRTF | n/a                                     | Cloud              | Achieves lowest time parameter                                    |
| 37       | SODA           | CyberShake workflow                     | IaaS Cloud         | Ensure deadline and minimize risk probability of tasks            |
| Proposed | EcoTaskSched   | COSCO, Docker, Python Tool, Azure Cloud | Cloud-fog          | Improve tasks scheduling, reduce energy footprints and ensure QoS |

Table 1. Comparison of traditional task scheduling algorithms and the ecotasksched model.

efficiency<sup>25,26</sup>. Here, we discuss some fundamental ML-based techniques, including artificial neural networks (ANNs), recurrent neural networks (RNNs), convolutional neural networks (CNNs), and reinforcement learning (RL), among others.

An ANN-based dynamic VM consolidation algorithm is proposed in<sup>38</sup> to optimize resources, reduce energy usage, and increase VM migration in a cloud environment. The proposed algorithm adaptively selects the best scheduling method based on cloud provider parameters and priority goals. The results demonstrate that, through higher VM migration, the proposed technique achieves better performance, minimizes energy consumption, and reduces SLA violations. The proposed model<sup>39</sup>, called the Cloud Service Orchestrator (CSO), employs cat-based swarm optimization for efficient resource management in data-intensive businesses to obtain optimal scheduling decisions and ensure QoS (e.g., energy efficiency, makespan) in the fog-cloud environment. The experiments are conducted using the COSCO<sup>40</sup> framework, considering thermal, energy, and cooling models to find the optimal solution for the multi-objective scheduling issue in both real-world and simulated cloud environments. The experimental results demonstrate that the proposed model outperforms state-of-the-art baseline models in terms of makespan, energy efficiency, and lower execution overhead in the fog-cloud environment.

Although CNNs are primarily used for visualization, they also have applications in cloud-based project design. CNNs can capture spatial patterns in data, which can be translated into meaningful resource management plans across nodes in a fog-cloud environment. By converting resource models into a 2D array format, CNNs can more effectively identify and predict task allocation, reducing energy consumption and improving efficiency. The proposed HUNTER scheduler uses a GGCN coupled with an RNN model to predict QoS parameters and make informed policy decisions that enhance performance and energy efficiency<sup>25</sup>. Similarly, the new HunterPlus scheduler extends the traditional GGCN model by utilizing a CNN scheduler, which outperforms the GGCN-based strategy in both job completion rates and energy usage per task. Specifically, it optimizes QoS metrics to make scheduling decisions more efficiently<sup>26</sup>. In contrast, RL model offers a different approach by treating the scheduling challenge as a sequential decision-making process. In RL, the agent learns to make decisions by interacting with the environment and receiving feedback in the form of rewards or punishments. This approach is particularly useful in dynamic and unpredictable environments, where the agent can adjust its strategy based on the consequences of its actions. An RL-based model called Weighted Double Deep Q-Network-based Reinforcement Learning (WDDQN-RL) has been proposed to schedule multiple workflows and optimize resources. This approach results in a shorter makespan and reduced costs by continuously learning from system performance<sup>41</sup>. One notable deep reinforcement learning (DRL) system, called DeepEE<sup>42</sup>, employs a Parameterized Action Space based Deep Q-Network (PADQN), which optimizes work scheduling and cooling control of data centers concurrently at multiple sites to achieve energy efficiency. The proposed scheduling technique improves IT performance, enhances energy savings in cooling systems, and ensures service quality in cloud computing. To address the safety evaluation issue in cloud data centers, a model-assisted DRL technique called LyaSafe<sup>43</sup> is proposed for cooling control to ensure compliance with strict safety standards. The primary objective of this work is to minimize carbon emissions and regulate the Rack Cooling Index (RCI) within a specified threshold. An end-to-end Deep Q Scheduler (DQS) task scheduling algorithm based on the DRL model is proposed in<sup>44</sup> to address high dimensional scheduling issues in cloud computing. The proposed model uses a CNN to extract task features and output information about the VMs executing the task in the cloud environment. This model enhances scheduling accuracy and minimizes task execution time. To address the Task as a Service (TaaS) scheduling problem, a DRL-based TaaS Cloud (DRLTC) scheduling is proposed<sup>45</sup> for the cloud environment. Tasks are scheduled based on their resource requirements and deadlines (time stamps). The proposed scheduling algorithm successfully schedules TaaS and achieves superior performance. A Hybrid ML technique called Resource Allocation Task Scheduling (RATS-HM)<sup>46</sup>, with combines efficient resource allocation with a security feature, is proposed for cloud computing. The proposed technique uses Improved Cat Swarm Optimization for Task Scheduling (ICSO-TS) with a short scheduler, which reduces execution time and increases system throughput in cloud computing. Additionally, a Group Optimization-based Deep Neural Network (GO-DNN) is used to optimize resource allocation, including load and bandwidth. Data encryption is applied to secure data storage. Simulation results show that the proposed RATS-HM provides security, minimizes makespan, and reduces energy consumption through efficient task scheduling in the cloud environment. In<sup>47</sup>, an automatic-scaling approach called Thoth is proposed, using Docker container technology for Platform as a Service (PaaS). Thoth automatically monitors resource utilization and dynamically allocates appropriate resources to each application. The Thoth algorithm is implemented using three approaches, namely Q-Learning, which optimizes computing resources more efficiently than rule-based and Neural Network (NN) algorithms. The authors conducted a systematic and comprehensive literature review to examine recent advancement in several approaches, including machine learning inference (MLI)<sup>48</sup>, cloud start latency<sup>49,50</sup>, function placement<sup>51</sup>, proactive content caching<sup>52</sup>, function offloading<sup>53</sup>, computation offloading<sup>54</sup>, and scheduling mechanisms<sup>55</sup> in a serverless computing environment. To handle the increasing number of modules that need to be executed on the Mobile Fog Computing (MFC), a Hidden Markov Model Auto-scaling Offloading (HMAO) method<sup>54</sup> is proposed to find the best place for module execution in order to minimize energy consumption, execution time; optimize the execution time, network resource usage, and delay of the modules on the mobile, Fog, or Cloud. The results obtained demonstrate that the proposed method is superior to the First-Fit (FF) and Q-learning methods. Table 2 summarizes recent research on ML-based techniques for optimizing resource allocation through different scheduling solutions in the cloud environment.

## Research problem

Although traditional task scheduling algorithms<sup>27–31,33–36</sup> have laid the foundation for business processes in cloud computing environments, they have notable limitations. In particular, they lack the ability to dynamically adapt to



| Refs.    | Technique                          | Evaluation tool                                               | Performance metrics                                       | Environment                 | Pros & Cons                                                                                                                                                       |
|----------|------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 25       | Hunter                             | CloudSim Toolkit, COSCO, Docker, Azure                        | Energy, SLA, waiting time, and migration time             | Fog-cloud                   | Achieve energy efficiency and improve performance<br>Dynamic workload challenge exists                                                                            |
| 26       | HunterPlus                         | COSCO framework/<br>Experimental                              | Energy, SLA, waiting time, and scheduling time            | Fog-cloud                   | QoS prediction, quickly adapts to environment, improved job completion rate, and minimize energy consumption<br>Limited consideration for heterogeneous resources |
| 38       | Dynamic VM consolidation algorithm | CloudSim Toolkit, PlanetLab VM Workload Trace, DeepLearning4j | Energy, SLA, Accuracy, Efficiency, and Response Time      | Critical data centers-cloud | Reduce SLA violation and energy usage<br>High initial training overhead<br>High computational cost                                                                |
| 39       | CSO                                | CloudSim Toolkit, COSCO Framework, Azure                      | Energy, SLA, QoS, makespan, and Execution Overhead        | Fog-cloud                   | Reduce energy usage, execution overhead, and execution time<br>Complex implementation                                                                             |
| 41       | WDDQN-RL                           | Python-based simulation, TensorFlow                           | Cost and makespan                                         | Cloud                       | Minimization of makespan and cost<br>Scalability challenges<br>Increased training time                                                                            |
| 42       | DeepEE                             | Computational Fluid Dynamics (CFD), Matlab, TensorFlow        | Energy, performance, Average Reward, Hotspot, and Quality | Cloud                       | Achieves energy saving, stable performance, and service quality<br>Computational overhead                                                                         |
| 43       | LyaSafe                            | EnergyPlus & thermodynamics                                   | Cooling, CO <sub>2</sub> , and safety                     | Data centers-cloud          | Ensure safety set and reduce carbon emissions                                                                                                                     |
| 44       | DQS                                | Deep Learning frameworks                                      | Accuracy and execution time                               | Multi-cloud                 | Enhance scheduling accuracy and minimize cost execution time<br>Computational complexity<br>High training overhead                                                |
| 45       | DRLTC                              | CloudSim                                                      | VM load-balancing                                         | Cloud                       | Improves performance<br>Slow convergence<br>Limited real-world validation                                                                                         |
| 46       | RATS-HM                            | CloudSim                                                      | Makespan, Energy, Bandwidth, Memory                       | Cloud                       | Optimize resource allocation, minimize makespan, reduce energy usage, and ensure security<br>Long training time<br>Scalability concerns                           |
| 47       | Thoth                              | Locust, Docker, Kubernetes, HAProxy, InfluxDB                 | Response time, request rate, service time, CPU            | Cloud                       | Save computing resources and ensure QoS<br>Computational cost                                                                                                     |
| 54       | HMAO                               | iFogsim                                                       | Energy, delay, and cost                                   | Mobile Fog Computing        | Optimize resource usage, minimize energy consumption, delay, and execution time of modules                                                                        |
| Proposed | EcoTaskSched                       | COSCO, Docker, Python, Azure fog and cloud                    | Energy, job completion, SLA, response time, throughput    | Fog-cloud                   | Improve tasks scheduling, reduce energy footprints and ensure QoS                                                                                                 |

**Table 2.** Comparison of ML-based models and the ecotasksched model.

changing workloads and do not incorporate predictive capabilities that optimize resource utilization and energy efficiency. For example, priority-aware and FCFS methods often fail to consider the characteristics of tasks and VMs, ultimately resulting in poor system performance. In contrast, ML-based scheduling models<sup>25,26,47,38,39, 41–46</sup> offer significant improvements over traditional scheduling methods for business processes by providing scalable, data-driven solutions capable of handling today's complex fog-cloud computing environments. However, ML-based scheduling models also have several deficiencies. ML-based algorithms are often trained on a manually selected set of features (labels) from sample data to design scheduling policies. Acquiring a large number of labeled samples is challenging in practice, and the quality of these features directly impacts the effectiveness of the scheduling algorithm. Additionally, most trained models are environment-specific, which can ultimately lead to system failure. For example, RL models are unable to adapt to hosts running at high capacity. Additionally, RL models face scalability issues, which lead to increased waiting times for resources in highly volatile environments. The main challenge for cloud service providers is efficiently allocating and utilizing cloud resources. These resources include VMs, CPUs, RAM, storage, bandwidth, and other components. Effective task scheduling techniques are required to manage these resources over the Internet in order to minimize energy consumption in cloud data centers. This in turn, helps reduce operational costs and environmental impact. On the other hand, from consumer's perspective, energy-efficient task scheduling techniques must meet the consumer's QoS requirements include response time, low makespan, and user satisfaction. The techniques must fulfill these demands stated in the SLA. Additionally, they should not impose any additional cost over the Internet and must be achieved in a dynamic fog-cloud environment. The aforementioned discussion motivates the authors to address these challenges, particularly the task scheduling problem, by proposing a suitable ML-based task scheduling solution in a dynamic fog-cloud environment. Moving forward, we will address the following research question: “How can ML-based models be integrated to optimize resource scheduling and energy consumption in fog-cloud environments?”

### Problem formulation

Cloud service providers strive to offer various cloud resources demanded by consumers to meet their QoS requirements such as execution time, response time, job completion rate, agreed-upon performance standards, as outlined in SLAs and throughput. Therefore, our main focus is to efficiently schedule each task to ensure successful completion with minimal energy consumption on the host systems. Hence, this can be achieved by

mathematically mapping the energy consumption of each scheduled task to the fog or cloud resource of each VM in the cloud data center. The objective function in Eq. (1) mathematically formulates this scenario. The variables (notations) and their meanings used throughout the article are listed in Table 3.

$$\text{Minimize } E_{\text{Total}} = \sum_{h=1}^{N_{\text{Host}}} \sum_{v=1}^{N_{\text{VM}}} \sum_{i=1}^N (E_{\text{fog}}(t_i) \cdot T_{\text{fog}}(t_i) \cdot R_{\text{fog}}(t_i) + E_{\text{cloud}}(t_i) \cdot T_{\text{cloud}}(t_i) \cdot R_{\text{cloud}}(t_i))_{h,v} \quad (1)$$

Subject to the following conditions:

$$R_{\text{fog}}(t_i) + R_{\text{cloud}}(t_i) = 1, \quad \forall i \in \{1, 2, \dots, N\} \quad (1a)$$

$$\frac{\sum_{i=1}^N J_{\text{completed}}(t_i)}{\sum_{i=1}^N J_{\text{assigned}}(t_i)} \geq J_{\text{min}} \begin{cases} 0 & \text{Job not completed} \\ 1 & \text{Job completed} \end{cases} \quad (1b)$$

$$\frac{\sum_{i=1}^N T_{\text{response}}(t_i)}{N} \leq T_{\text{max}} \quad (1c)$$

$$\frac{\sum_{i=1}^N \text{SLA}_{\text{violation}}(t_i)}{\sum_{i=1}^N J_{\text{total}}(t_i)} \leq \text{SLA}_{\text{max}} \begin{cases} 0 & \text{SLA not violated} \\ 1 & \text{SLA is violated} \end{cases} \quad (1d)$$

The objective function in Eq. (1) aims to minimize the overall energy consumption of each task ( $t$ ) when allocated, scheduled, and executed on appropriate VM ( $v$ ) hosted on a host ( $h$ ), while satisfying all the conditions outlined in Eqs. (1a)-(1d) in a fog-cloud of things environment. The condition in Eq. (1a) states that each task ( $t$ ) is allocated to exactly one resource ( $R$ ), either to the fog ( $R_{\text{fog}}$ ) or the cloud ( $R_{\text{cloud}}$ ). Equation (1b) represents the job completion rate, where the ratio of completed jobs ( $J_{\text{completed}}$ ) to the total assigned jobs ( $J_{\text{assigned}}$ ) must meet the minimum job completion rate ( $J_{\text{min}}$ ). Equation (1c) states that the average response time ( $T_{\text{response}}$ ) of all tasks should not exceed the maximum threshold response time ( $T_{\text{max}}$ ). Finally, Eq. (1d) indicates that the SLA violation rate ( $\text{SLA}_{\text{violation}}$ ) for all tasks ( $J_{\text{total}}$ ) processed by the system should not exceed the maximum threshold of the agreed-upon performance standards ( $\text{SLA}_{\text{max}}$ ). While Eq. (1) has several real-world implications and benefits in dynamic fog-cloud computing environments. First, cloud service providers will be able to reduce operational costs by lowering electricity expenses. Second, service user satisfaction can be improved by ensuring the SLAs are met. Third, it helps balance workloads through efficient resource utilization across fog and cloud resources, preventing system overloads and failures. Finally, it enhances job completion rates and response times for real-time IoT applications like smart grids, smart cities, and healthcare.

| Variables                       | Meaning                                                           |
|---------------------------------|-------------------------------------------------------------------|
| $E_{\text{Total}}$              | Total energy consumption of $N$ tasks                             |
| $N_{\text{Host}}$               | Total number of hosts                                             |
| $N_{\text{VM}}$                 | Total number of VMs on a host $h$                                 |
| $N$                             | Total number of scheduling tasks on a VM ( $v$ )                  |
| $t_i$                           | task $i$ on the cloud or fog node                                 |
| $E_{\text{fog}}$                | Energy consumption of task $i$ on the fog node                    |
| $T_{\text{fog}}$                | Makespan of task $i$ on the fog node                              |
| $E_{\text{cloud}}$              | Energy consumption of task $i$ on the cloud node                  |
| $T_{\text{cloud}}$              | Makespan of task $i$ on the cloud node                            |
| $h$                             | Host                                                              |
| $T_{\text{response}}$           | Response time of a task                                           |
| $J_{\text{completed}}$          | Job completed successfully                                        |
| $J_{\text{min}}$                | Minimum job completion rate                                       |
| $J_{\text{assigned}}$           | Job assigned to the system                                        |
| $J_{\text{total}}$              | Total jobs processed                                              |
| $R_{\text{fog}}$                | Binary variable to show that task is assigned to a fog resource   |
| $R_{\text{cloud}}$              | Binary variable to show that task is assigned to a cloud resource |
| $\text{SLA}_{\text{violation}}$ | SLA violation of a job                                            |
| $\text{SLA}_{\text{max}}$       | Maximum SLA threshold                                             |
| $\forall$                       | For all                                                           |
| $\in$                           | Belongs to                                                        |
| $T$                             | Time steps                                                        |
| $D$                             | Decision                                                          |

**Table 3.** Variables and its meaning.

In contrast to previous research, our proposed EcoTaskSched model introduces a novel solution. Its uniqueness lies in its ability to find an optimal solution to the research problem. It achieves this by efficiently scheduling tasks. The main goal is to minimize overall energy usage. At the same time, it ensures that QoS factors are satisfied in the underlying computing environment. The next section outlines the integration of ML-based models with the system architecture.

## Overview of ML-based models and the system architecture

This section introduces the proposed EcoTaskSched model, detailing its innovative integration of a Bidirectional Long Short-Term Memory (BiLSTM) network with a Convolutional Neural Network (CNN) model. The main objective is to optimize task scheduling for energy efficiency while maintaining QoS in fog-cloud computing environments.

### Bidirectional long short-term memory (BiLSTM)

The proposed EcoTaskSched approach employs a DNN-based BiLSTM network. This network is an extension of the unidirectional LSTM. It processes complex input sequences such as temporal patterns, for bidirectional analysis in task scheduling data. It offers long-term temporal dependency insights that contribute to energy savings, provide excellent accuracy, and ensure QoS provisions in fog-cloud networks. This new extension scheme leverages the capabilities of LSTM by considering both forward (past to future) and backward (future to past) data dependencies. It captures long-term dependencies more effectively, leading to improved task scheduling predictions. This dual evaluation model captures more complex contexts and nuances in the data, such as sequences of time steps. It analyzes information from both directions, which is essential for making accurate scheduling decisions. For instance, interdependencies that might be missed when analyzed unidirectionally can be effectively detected. The proposed EcoTaskSched model processes data in two phases. This allows it to accurately identify dependencies and assign tasks for better distribution. As a result, it reduces energy consumption while optimizing QoS. In general, BiLSTM operates in two directions: forward LSTM and backward LSTM, as mathematically expressed below.

#### Forward LSTM

Therefore, for a given input sequence  $x_t = \{x_1, x_2, \dots, x_T\}$ , at each time step  $t = \{1, 2, \dots, T\}$ , the hidden state update ( $h_t^{\rightarrow}$ ) in the forward LSTM gates (forget, input, cell state update, and output) can be computed using standard Eqs. (2)–(7) as follows:

$$f_t^{\rightarrow} = \sigma(W_f \cdot [h_{t-1}^{\rightarrow}, x_t] + b_f) \quad (2)$$

$$i_t^{\rightarrow} = \sigma(W_i \cdot [h_{t-1}^{\rightarrow}, x_t] + b_i) \quad (3)$$

$$\bar{c}_t^{\rightarrow} = \tanh(W_c \cdot [h_{t-1}^{\rightarrow}, x_t] + b_c) \quad (4)$$

$$c_t^{\rightarrow} = f_t^{\rightarrow} \odot c_{t-1}^{\rightarrow} + i_t^{\rightarrow} \odot \bar{c}_t^{\rightarrow} \quad (5)$$

$$o_t^{\rightarrow} = \sigma(W_o \cdot [h_{t-1}^{\rightarrow}, x_t] + b_o) \quad (6)$$

$$h_t^{\rightarrow} = o_t^{\rightarrow} \odot \tanh(c_t^{\rightarrow}) \quad (7)$$

where, Eq. (2) expresses the forget gate ( $f_t^{\rightarrow}$ ) which identifies which future information should be removed from the cell state. Equation (3) defines the input gate activation ( $i_t^{\rightarrow}$ ) which regulates which new information should be incorporated into the cell state. Equation (4) represents candidate cell state ( $\bar{c}_t^{\rightarrow}$ ) which generates a potential new cell state update. Equation (5) defines the cell state update ( $c_t^{\rightarrow}$ ) which integrates the previous cell state with the update cell state to store relevant information, whereas the output gate activation ( $o_t^{\rightarrow}$ ) is expressed in Eq. (6) which controls what part of the cell state should be exposed as the hidden state. The final updated hidden state ( $h_t^{\rightarrow}$ ) is formulated in Eq. (7) of the forward LSTM computation, which captures and carries temporal dependencies in the output. Therefore, the forward LSTM learns temporal dependencies by processing sequential tasks and resource metrics over time to optimize task allocation. It improves decision-making for energy-efficient task scheduling by leveraging features extracted from a CNN in a fog-cloud environment.

#### Backward LSTM

Similarly, the hidden state update ( $h_t^{\leftarrow}$ ) in the backward LSTM for each time step ( $t$ ) processes the input sequence in reverse order from ( $t = \{T, \dots, 1\}$ ), where each gate can be computed using the standard Eqs. (8)–(13) as follows:

$$f_t^{\leftarrow} = \sigma(W_f \cdot [h_{t+1}^{\leftarrow}, x_t] + b_f) \quad (8)$$

$$i_t^{\leftarrow} = \sigma(W_i \cdot [h_{t+1}^{\leftarrow}, x_t] + b_i) \quad (9)$$

$$\bar{c}_t^{\leftarrow} = \tanh(W_c \cdot [h_{t+1}^{\leftarrow}, x_t] + b_c) \quad (10)$$

$$c_t^{\leftarrow} = f_t^{\leftarrow} \odot c_{t+1}^{\leftarrow} + i_t^{\leftarrow} \odot \bar{c}_t^{\leftarrow} \quad (11)$$

$$o_t^{\leftarrow} = \sigma(W_o \cdot [h_{t+1}^{\leftarrow}, x_t] + b_o) \quad (12)$$

$$h_t^{\leftarrow} = o_t^{\leftarrow} \odot \tanh(c_t^{\leftarrow}) \quad (13)$$



where, Eq. (8) expresses the forget gate ( $f_t^{\leftarrow}$ ) which identifies which previous (past) information should be removed in the reverse sequence. Equation (9) defines the input gate activation ( $i_t^{\leftarrow}$ ) to control how much new information should be retained. Equation (10) represents the candidate cell state ( $\tilde{c}_t^{\leftarrow}$ ) which generates a new potential cell state update and Eq. (11) defines the cell state update ( $ch_t^{\leftarrow}$ ) which updates the stored information using the past cell state and candidate update. While the output gate activation ( $o_t^{\leftarrow}$ ) is expressed in Eq. (12) decides which part of the cell state should contribute to the hidden state. The final update hidden state ( $h_t^{\leftarrow}$ ) is formulated in Eq. (13) which produces the final hidden state for the backward pass. In addition, the variables  $W_f, W_i, W_c, W_o$  represent the weight matrices for the input sequence ( $x_t$ ), and  $b_f, b_i, b_c, b_o$  represent bias terms. The sigmoid function ( $\sigma$ ) is used to map input ( $x$ ) to value between 0 and 1 as expressed in Eq. (14):

$$\sigma = \frac{1}{1 + e^{-x}} \quad (14)$$

While  $\odot$  is used for element-wise multiplication. The final output ( $h_t$ ) of BiLSTM is an integration of the forward ( $h_t^{\rightarrow}$ ) and backward ( $h_t^{\leftarrow}$ ) LSTMs for richer temporal learning and enhancing accuracy in predicting optimal task allocation, as expressed in Eq. (15):

$$h_t = [h_t^{\rightarrow} || h_t^{\leftarrow}] \quad (15)$$

where, hidden state  $h_t \in R^n \times R^n = R^{2n}$  if the forward ( $h_t^{\rightarrow}$ ) and backward ( $h_t^{\leftarrow}$ ) states have  $n$  units. The symbol  $R$  represents a real-valued  $2n$ -dimensional vector space ( $h_t$ ) at each time step  $t$ . If a fully connected layer of a neural network follows the BiLSTM, then the softmax function is applied for classification and making predictions in task scheduling of fog-cloud of things environment, as mathematically presented in Eq. (16) below:

$$Output_t = softmax(W \cdot h_t + b) \quad (16)$$

where, Eq. (17) formulates the softmax function as:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (17)$$

where,  $x_i$  denotes  $i$ -th element in the input vector space  $x$ . The backward LSTM operates on a feature set extracted through a CNN model and processes input sequences in reverse order to capture past dependencies, enhancing accuracy in predicting optimal task allocation and aiding in energy-efficient task scheduling in fog-cloud environments.

### CNN-based task scheduling

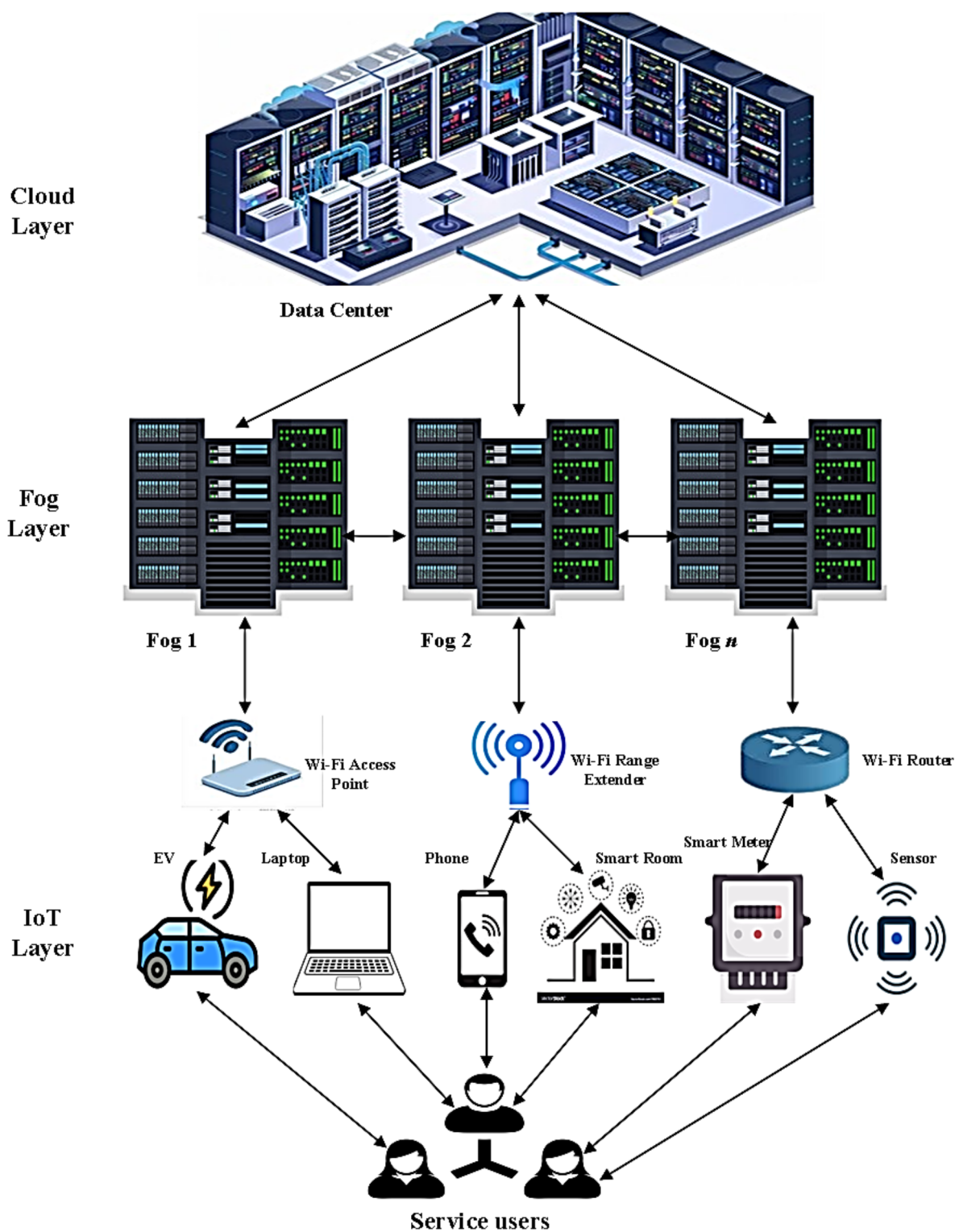
The proposed EcoTaskSched model incorporates CNN-based task scheduling, enhancing resource allocation and energy efficiency in a fog-cloud computing environment. The model leverages the power of CNNs to process and interpret resource utilization data, making scheduling decisions more efficient and effective. Generally, CNNs are a type of deep learning paradigm used primarily to process grid-like data structures, such as images. They consist of convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply filters to the input data to detect patterns, while pooling layers reduce the dimensionality of the data while preserving important features. The fully connected layers then utilize these features for classification or prediction services. In tasks planning, CNNs analyze tasks and predict the optimal allocation of resources. Resource utilization metrics, such as CPU, memory, and bandwidth usage, are organized into a 2D matrix or 'picture' that represents the current state of the system. This enables CNNs to process data graphically, identify patterns, and discover connections that inform policy decisions. Equation (18) formulates CNN-based task scheduling by analyzing task characteristics and resource usage patterns in a fog-cloud of things environment.

$$x_{i,j} = CNN(t_i, R_j) \quad \forall i \in \{1, 2, \dots, N\}, \forall j \in \{1, 2, \dots, M\} \quad (18)$$

Here,  $x_{i,j}$  is a binary output decision variable, where  $x_{i,j} = 1$  indicates that task  $t_i$  is allocated to exactly one resource  $R_j$ . Otherwise,  $x_{i,j} = 0$  means that the task is not assigned to any resource in the fog or cloud. The variable  $M$  represents a 2D matrix (in array form) of current resource usage states (CPU, memory, and bandwidth, etc.), which the CNN model processes for each task-resource pair to produce a decision  $x_{i,j}$ . This decision is then mapped to the energy usage metric.

### System architecture

This section outlines the system architecture of the proposed EcoTaskSched approach, highlighting its distinct N-tier layered structure, which consists of three main layers: the IoT layer, the fog layer, and the cloud layer. At the bottom of the hierarchical system architecture is the front-end layer, which consists of numerous IoT devices (e.g., sensors, smartphones, Wi-Fi router, smart meters, etc.). Above it is the fog layer, also known as the middle tier, which provides fog computing services. The top layer is the cloud layer, which hosts multiple technologies, offers services (e.g., IaaS, PaaS, and SaaS), and provides additional third-party products to end users, as illustrated in Fig. 1. The underlying system architecture integrates advanced ML-based techniques for energy-efficient task scheduling and resource management. The primary goal is to minimize energy consumption while ensuring



**Fig. 1.** System architecture of the EcoTaskSched model.

QoS parameters including response time, job completion rate, and other relevant factors in a fog-cloud of things environment.

#### *IoT layer*

This layer comprises IoT devices such as smartphones, tablets, laptops, smart meters, and EVs, as well as other edge devices like gateways, and Wi-Fi routers, which have limited computational capabilities (e.g., memory, processing, and storage etc.). It collects large volumes of data generated by these IoT devices and sends it to

the fog node (broker) in the next layer of the hierarchical system architecture for further processing. This layer serves as the front end, allowing service users to interact with the fog-cloud architecture.

#### Fog layer

The fog layer, also known as the micro data center, distributes computing resources and services by bringing cloud services closer to the IoT layer. It includes the fog broker, which manages heterogeneous incoming tasks and allocates them to the appropriate fog resources. The key modules in this layer include the Service Manager, Resource Manager, and Data Center Manager. The Service Manager handles heterogeneous services and processes tasks. Additionally, it comprises two sub-modules: the SLA Manager, which manages the SLA between service users and the service provider based on QoS requirements, and the QoS Manager, which ensures that tasks meet the specified QoS parameters. The Resource Manager includes the EcoTaskSched scheduler, which orchestrates tasks based on resource metrics provided by the resource monitor. It leverages a CNN-based surrogate model to estimate QoS parameters and dynamically adapt to environmental changes. Similarly, the Data Center Manager monitors resource utilization and QoS parameters while managing task allocation and migration between cloud and fog nodes.

#### Cloud layer

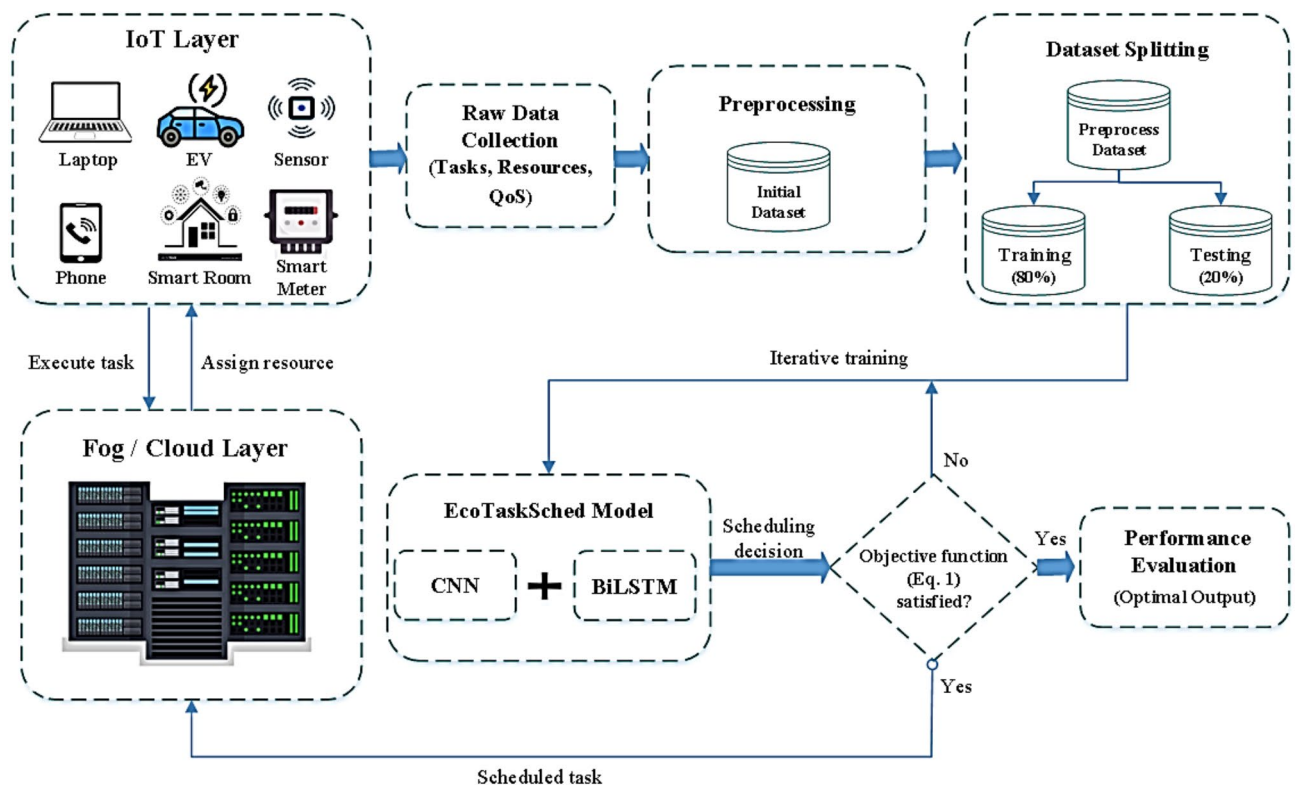
The upper layer of the hierarchical system architecture offers services such as software, infrastructure, and platforms, along with a vast pool of heterogeneous resources. These resources include fog resources (low latency, resource-constrained) and cloud resources (high capacity, geographically distant), which are accessible via the Internet anytime, anywhere. This layer consists of highly virtualized data center hosts with advanced computing capabilities, where tasks are executed and processed. The cloud layer ensures that tasks are allocated to the most suitable resources, optimizing performance and energy efficiency.

### Proposed ecotasksched methodology

This section provides detailed insights into the proposed EcoTaskSched model for energy-efficient task scheduling in a fog-cloud of things environment, as illustrated in Fig. 2. The overall approach is divided into subsections, including an overview of the dataset, raw data collection, data preprocessing, the training and testing setup, the proposed task scheduling algorithm, and performance evaluation metrics. The primary aim of the proposed model is to solve the objective function, i.e., the research problem discussed in this article. The subsections below provide a detailed description of the aforementioned task scheduling challenge.

#### Overview of the dataset

To evaluate the performance of the EcoTaskSched model, it is essential to generate and preprocess a suitable dataset that accurately reflects the dynamic and heterogeneous nature of fog-cloud computing. The methodology



**Fig. 2.** Schematic diagram of the proposed EcoTaskSched model.

for dataset generation and preprocessing involves several key steps. The dataset is generated using a real fog-cloud environment setup, leveraging the COSCO framework. This framework simulates a heterogeneous, distributed fog-cloud environment, providing a realistic scenario for testing task scheduling algorithms. Four standard Azure B2s virtual machines (VMs) are deployed in the UK region, each with 2 cores and 4 GB of RAM. These VMs represent the cloud resources in the computing environment. This research project is based on the DeFog benchmark<sup>36</sup>, which includes diverse and complex AI workloads using Docker containers. While tasks are generated following a Poisson distribution, with an average of 2 tasks per interval for normal workloads and 4 tasks per interval for intensive workloads. The scheduling interval is set to 50 each with 300 s to allow sufficient time for the processes. Additionally, each test is conducted over 20 runs to collect and validate sufficient QoS metrics.

### Raw data collection

In this step, for each configured test, performance evaluation metrics such as energy consumption, response time, job completion rate, and SLA violations are recorded, including CPU usage, RAM usage, Disk\_IO, Network\_IO, and the number of active containers per host. Additionally, two separate datasets are generated to train the model: one for normal performance conditions (with 2 tasks per interval) and another for intensive performance conditions (with 4 tasks per interval). This approach ensures that the proposed model is trained under diverse workload conditions, optimizing its robustness and scalability.

### Dataset preprocessing

The raw data collected needs to be normalized to ensure all factors are on a consistent scale, facilitating more efficient learning by ML-based models. For example, CPU, RAM, Disk\_IO, and Network\_IO usage values are normalized to a range of 0 to 1. In addition, new features have been developed to capture relationships and dependencies between services and objects. One-hot encoding is applied to categorical variables, such as host IDs and identifiers of various objects. Furthermore, data enhancement techniques are employed to improve the robustness of proposed model, which can also be used to generate composite metrics as they are combined. This process involves slightly perturbing the original data to generate artificial data points, plotting work occurrence patterns, and incorporating resource information. Robust data generation and preprocessing using the DeFog benchmark ensure that the EcoTaskSched model is trained on a realistic and randomly generated dataset. The dataset size is set to 1,000 samples i.e. a random feature matrix with a shape of (1000, 10). Each row represents a task's resource consumption, such as CPU usage, disk, RAM, network bandwidth, and while each column represents a feature. Further, random binary labels (0 or 1) are generated for each task, where 0 represents fog resources and 1 represents cloud resources. A MinMaxScaler is used to scale the features, transforming them into a normalized range of [0, 1]. This enhances the proposed model's ability to make accurate task allocation and scheduling decisions, optimize resource utilization, and minimize energy consumption in fog-cloud computing environments.

### Dataset splitting

In this step, we rigorously evaluate the performance of the EcoTaskSched model using a comprehensive training and testing setup. The preprocessed dataset is divided into training and testing sets. Typically, the proposed model is trained on 80% of the data, while the remaining 20% is reserved for testing and validation. This setup ensures that the model is trained on realistic data, evaluated under various scenarios, and benchmarked against baseline models to validate its effectiveness in energy-efficient task scheduling.

#### *Training setup*

The EcoTaskSched model starts with a CNN that processes tasks and resource metrics to extract feature sets. The architecture of the CNN consists of several convolutional layers to capture spatial hierarchies in the input data followed, by fully connected layers. The model is trained on a training feature set ( $X_{train}$ ) and training labels ( $y_{train}$ ), which include binary classification: 0 = fog, 1 = cloud. The proposed model uses a Keras Sequential API to define a deep learning model. To extract spatial patterns from input sequences, a Convolutional layer (Conv1D) are applied. It uses 64 filters for feature extraction and applies a convolution filter of size 3. This layer uses ReLU activation to introduce non-linearity and accepts sequential input or time-series data. Further, a flatten layer is used to convert multi-dimensional feature maps into a 1D array for LSTM processing. BiLSTM Layers (64 units and 32 units) are used to capture temporal dependencies in both forward and backward directions. Further, fully connected layers (Dense) with 16 neurons and ReLU activation are used for non-linearity. Similarly, an output layer with 1 neuron and sigmoid activation is applied for binary classification (i.e. 0 = Fog, and 1 = Cloud). The model is compiled by using the Adam optimizer, known for its efficiency with large datasets and effective handling of noisy gradients. The loss function used is binary cross-entropy, which designed for binary classification. Key hyperparameters, such as batch size, and the number of epochs, have been adjusted to improve the model's performance. For example, a learning rate of 0.001, a batch size of 32 (i.e. processing 32 samples at a time), 100 epochs (i.e. training for 100 iterations) and a verbose value of 1 (displaying training progress) are commonly used. The preprocessed data is divided into training and testing sets, with 80% of the data used for training. The training data includes various metrics such as energy consumption, CPU, RAM, Disk\_IO, Network\_IO, and SLA violations etc. During training, the model learns to predict optimal task allocation by minimizing the loss function. The CNN model works in tandem with the BiLSTM, which extracts spatial features from system state, such as the available resources and the current distribution of tasks among virtualized hosts. The BiLSTM involves forward and backward passes, allowing it to analyze a sequence of past system states. It is capable of remembering spikes in resource usage and predicting future demands for CPU, memory, and network bandwidth across multiple hosts in datacenters, which are updated at regular scheduling

intervals. By training the BiLSTM on these sequences of historical data, the scheduler learns from past system states, enabling it to decide where to allocate or migrate tasks during both peak and low-usage hours based on the real-time state of the host systems.

#### *Testing setup*

The remaining 20% of the preprocessed dataset is used for testing. This data is not seen by the model during training, ensuring an unbiased evaluation of its performance. Several metrics are used to assess the model's effectiveness. The total energy consumed per completed task is measured in kWh. The ratio of completed jobs to assigned jobs indicates the model's efficiency in handling task workloads. The number of SLA violations reflects the model's ability to meet QoS requirements. The time taken to complete each task is measured in seconds. The performance of the EcoTaskSched model is compared against baseline models such as the traditional GGCN<sup>16</sup> and BiGGCN<sup>18</sup>. These comparisons highlight the improvements achieved by incorporating CNN with BiLSTM in the EcoTaskSched model.

#### **Proposed ecotasksched algorithm**

This section presents Algorithm 1, which illustrates the working process of the proposed EcoTaskSched model. The model schedule tasks using a CNN with a BiLSTM network, which is deployed on VMs in the host data center of a fog-cloud environment. The primary goal is to solve the objective function (Eq. (1)) along with its operational constraints (Eqs. (1a)–(1d)). The pseudocode leverages the integration of IoT with a fog-cloud architecture and employs ML-based models to efficiently schedule tasks. This approach helps reduce overall energy consumption while ensuring QoS requirements within the system's layered architecture.



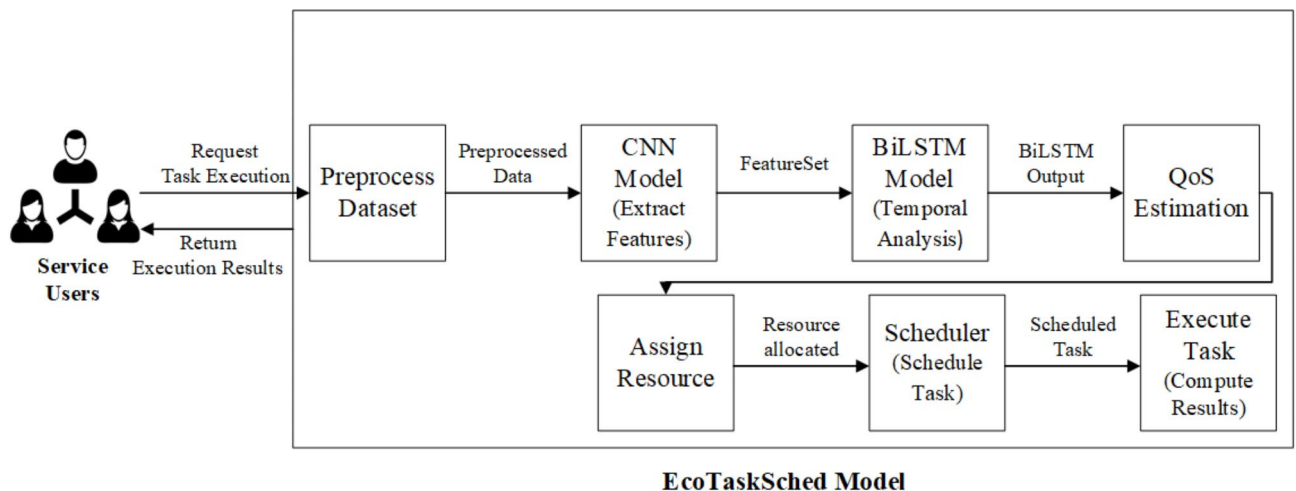
|               |                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Begin</b>  |                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Step 1</b> | <b>Input</b>                        | $t = \{t_1, t_2, \dots, t_N\}, I_t = 50, F = \{F_1, F_2, \dots, F_N\}, C = \{C_1, C_2, \dots, C_N\}, T_{assigned}, QoS = \{SLA_{violation}, T_{response}, J_{completed}\}, R = \{CPU, RAM, Disk\_IO, Network\_IO\}, E_{Threshold}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Step 2</b> | <b>Initialize COSCO environment</b> | <b>Load</b> Fog-Cloud configuration using <b>Table 5</b><br><b>Initialize</b> Fog ( $F$ ) and Cloud ( $C$ ) resources (VMs, Nodes, $Network\_IO$ etc.)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Step 3</b> | <b>DeFog benchmark Dataset</b>      | <b>Set</b> Dataset_size = 1000<br><b>Extract</b> Feature Set ( $R = \{CPU, RAM, Disk_{IO}, Network_{IO}\}, t$ )<br><b>Set</b> Labels (Fog=0, Cloud=1, Dataset_size)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Step 4</b> | <b>Preprocessing</b>                | <b>Normalize</b> MinMaxScaler (Feature Set)<br><b>Split</b> Dataset into training ( $X_{train}, y_{train}$ ) and testing sets ( $X_{test}, y_{test}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Step 5</b> | <b>Train CNN-BiLSTM</b>             | <b>Initialize</b> Sequential model<br><b>Use</b> $F_{spatial}$ = CNN (Feature Set) using <b>Eq. (18)</b><br><b>Use</b> $F_{temporal}$ = BiLSTM (FeatureSet) based on <b>Eq. (16)</b><br><b>Concatenate</b> Sequential model = [ $F_{spatial}; F_{temporal}$ ]<br><b>Compile</b> Adam optimizer(Sequential model)<br><b>Train</b> Sequential model ( $X_{train}, y_{train}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Step 6</b> | <b>Task Scheduling Loop</b>         | <b>Procedure</b> Surrogate Model (Sequential model, $F, C, X_{test}$ )<br>$T_{assigned} = []$<br><b>for</b> $t_i$ in scheduling interval ( $I_t$ ) <b>do</b><br>$t_i$ = Reshape for prediction ( $t_i$ , axis=0)<br>$D$ = Predict ( $t_i$ )<br><b>if</b> $D > 0.5$ <b>then</b><br>Assign $t_i \rightarrow C_i$<br><b>else</b><br>Assign $t_i \rightarrow F_i$<br><b>end if</b><br><b>if</b> $t_i == Fog(F_i)$ && $F_i[VMs'] > 0$ <b>then</b><br>$F_i[VMs'] -= 1$<br>Append $T_{assigned}(t_i, F_i)$<br><b>elseif</b> $t_i == Cloud(C_i)$ && $C_i[VMs'] > 0$ <b>then</b><br>$C_i[VMs'] -= 1$<br>Append $T_{assigned}(t_i, C_i)$<br><b>end if</b><br>Total time += clock.tick<br><b>end for</b><br><b>Return</b> $T_{assigned}$ , Total time                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Step 7</b> | <b>Performance Evaluation</b>       | <b>for</b> $i = \{1, 2, \dots, N\}$ <b>do</b><br><b>L: Compute</b> $E_{consumption}(t_i) = EnergyConsumption(t_i, R_i)$<br><b>Compute</b> $T_{response}(t_i) = ResponseTime(t_i, R_i)$<br><b>Compute</b> $SLA_{violation}(t_i) = SLAViolation(t_i, QoS_i)$<br><b>if</b> $E_{consumption}(t_i) < E_{Threshold}$ && $T_{response}(t_i) \leq T_{max}$ && $SLA_{violation}(t_i) = False$ <b>then</b><br>Assign $t_i \rightarrow F_i$<br>$a++$<br><b>Execute</b> ( $t_i, F_i$ )<br><b>if</b> $Job_{completed}(t_i) = 1$ <b>then</b><br>$c++$<br><b>end if</b><br><b>else</b><br>Assign $t_i \rightarrow C_i$<br>$a++$<br><b>Execute</b> ( $t_i, C_i$ )<br><b>if</b> $Job_{completed}(t_i) = 1$ <b>then</b><br>$c++$<br><b>end if</b><br><b>end if</b><br><b>Monitor</b> ( $t_i, QoS_i$ )<br><b>if</b> $QoS_i > SLA_{max}(t_i)$ && $\frac{c}{a} < Job_{min}$ <b>then</b><br><b>goto</b> L<br><b>end if</b><br>$E_{Total} += E_{consumption}(t_i)$<br>$Cost_{compute} = \sum_{i=1}^N (\frac{t_i}{60} \times VM.P_i)$<br>$Throughput = \frac{\sum J_{completed}}{Total\ time}$<br><b>end for</b><br><b>Update</b> COSCO simulation with scheduled task placement<br><b>Store</b> simulation results for analysis<br><b>Compare</b> simulation results against Hunter and HunterPlus |
| <b>Step 8</b> | <b>Output</b>                       | $E_{Total}, E_{consumption}, c, a, QoS, T_{response}, SLA_{violation}, Cost_{compute}, Throughput$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>End</b>    |                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

**Algorithm 1.** coTaskSched for energy-efficient task scheduling using CNN with BiLSTM

Algorithm 1 completes in eight steps to implement the EcoTaskSched model, aiming to satisfy the objective function given in Eq. (1), which focuses on reducing total energy consumption while ensuring QoS provisioning. In Step 1, input data is collected for the algorithm, including the incoming tasks list ( $t$ ) from IoT devices; task attributes such as size, type and priority, which are used in the CNN feature extraction process; the nodes list (fog ( $F$ ) and cloud ( $C$ )); the resource metrics list ( $R$ ), including CPU, RAM, Disk\_IO, and Network\_IO; scheduling interval ( $I_t$ ), the list of QoS parameters ( $QoS$ ), consisting of  $SLA_{violation}$ ,  $T_{response}$ , and  $J_{completed}$ ; list of assigned resources ( $T_{assigned}$ ) and the energy threshold ( $E_{Threshold}$ ). In Step 2, the COSCO framework is initialized with fog and cloud resources and configurations are loaded, as defined in Table 5. In Step 3, the DeFog benchmark is used to generate and load the dataset performing resource workload feature extraction and binary labelling. The feature set consisting of resources metrics ( $R$ ) and task details ( $t$ ) across all hosts is preprocessed by removing noise and missing values, creating a normalized dataset from the collected raw data ( $t$ ,  $R$ ) in Step 4. This preprocessing is essential for the BiLSTM and CNN models. The normalized dataset is divided into training (80%) and testing (20%) sets. Next, a CNN model (expressed in Eq. (18)) is applied to the resource metrics ( $R$ ) to extract spatial features ( $F_{spatial}$ ) and identify patterns helpful in making scheduling decisions. A BiLSTM model (expressed in Eqs. (2)–(17)) is applied to the extracted features to capture temporal dependencies and dynamics ( $F_{temporal}$ ) from historical tasks and resource data. The spatial features and temporal features are concatenated into a sequential model. The model is compiled using Adam optimizer and trained with historical workload data. In Step 6, a task scheduling loop is started by applying a surrogate model which retrieve and predicts tasks allocation and system resource status in each scheduling interval ( $I_t$ ) using trained CNN-BiLSTM model. It dynamically assigns tasks to Fog or Cloud based on resource availability. In Step 7, performance of the scheduling model is evaluated. First, the energy consumption, response time, SLA violation for each executing task are estimated on the assigned resource. The execution cost and throughput are calculated as well. The computed values are then checked to determine whether the task should be assigned, scheduled, and executed on the selected fog or cloud node, ensuring compliance with Eq. (1a) and Eq. (1c). Similarly, if a job (task) is successfully completed, its QoS parameters are monitored to ensure that sub conditions (1b) and (1d) of the objective function are satisfied. If the SLA is violated and the minimum job completion rate requirements etc. are not met, the scheduling process loops and adjusts for each task using the *goto* statement. Further, the COSCO simulated framework is updated with scheduled task placement and results are stored for future analysis. The results are compared against the baseline models (Hunter and HunterPlus) for performance evaluation of the proposed model. Finally in Step 8, Algorithm 1 returns the total energy consumption ( $E_{Total}$ ), the energy consumption ( $E_{consumption}$ ) of each task, execution cost, the total jobs completed ( $c$ ), the total jobs assigned ( $a$ ), the QoS parameters, including  $T_{response}$ ,  $SLA_{violation}$ , and throughput, as output. The simulation ends and generate final performance report. Algorithm 1 concludes by successfully solving the objective function, i.e., all tasks are scheduled in an energy-efficient manner using ML-based models (CNN with BiLSTM) while minimizing overall energy consumption and ensuring an acceptable response time, job completion rate, SLA compliance, execution cost, and throughput in the fog-cloud computing environment.

### Overall time complexity

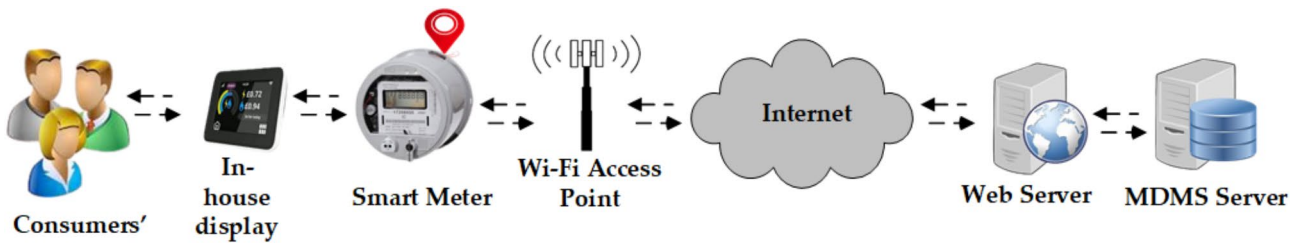
The process of the proposed EcoTaskSched model is implemented in Algorithm 1. The time complexity can be analyzed based on the key steps as follows: Step 2 and 3 takes  $O(1)$  to initialize the simulated environment and load the DeFog benchmark dataset. Step 4 has a time complexity of  $O(n)$ , where  $n$  depends on the dataset size for normalization and preprocessing. The time complexity for training the CNN-BiLSTM model is  $O(n \log n)$  due to the dominant factors i.e. CNN convolution and BiLSTM feature extraction operations in Step 5. In step 6, task scheduling loop has a time complexity of  $O(m)$  where  $m$  is the number of incoming tasks. Similarly, the performance evaluation step has a time complexity of  $O(m)$ . Overall, the time complexity of Algorithm 1



**Fig. 3.** Sequence diagram of the proposed EcoTaskSched model.

| IoT device                    | Application                             | Traffic type           | Sampling frequency  | Data generation rate |
|-------------------------------|-----------------------------------------|------------------------|---------------------|----------------------|
| Smart Lights                  | Automated lighting control              | Periodic               | Every few seconds   | Low (few KB)         |
| Surveillance Cameras          | CCTV (Security monitoring)              | Continuous             | Live streaming      | High (MB-GB)         |
| Smart Meters                  | Energy usage monitoring                 | Periodic               | Every hour          | Medium (KB-MB)       |
| IoT-enabled Projectors        | Smart classroom projection control      | On lecture start       | Event-based         | Medium (MB)          |
| Fire & Smoke Detectors        | Fire hazard detection                   | Event-based            | On hazard detection | Low (KB)             |
| Wi-Fi Access Points & Routers | Provides internet connectivity          | Continuous             | Always active       | High (MB-GB)         |
| Smart Parking Sensors         | Detects available parking spots         | Periodic               | Every few minutes   | Low (KB)             |
| Smart Phones                  | Internet access, calls, messaging, apps | Continuous/Event-based | Always active       | High (MB-GB)         |
| Laptops                       | Work, online classes, research          | Continuous/Event-based | Always active       | High (MB-GB)         |

**Table 4.** IoT devices inside a smart building along with their traffic details.



**Fig. 4.** Case study example of a real-world application.

is  $O(n \log n) + O(m) \approx O(n \log n)$  for training and  $O(m)$  for task scheduling. Figure 3 illustrates the flow chart (sequence diagram) of Algorithm 1, detailing the steps (activities) of the proposed model.

**Case study**

The real-world case study includes thousands of IoT-enabled devices, such as mobile phones, Wi-Fi access points, routers, smart lights, surveillance cameras, smart meters, and other connected devices, along with their traffic details as listed in Table 4 inside a smart building on a university campus. The IoT devices in the system architecture require heterogeneous communication support, low-latency response, real-time processing, optimized bandwidth allocation, load balancing, and energy efficiency in a fog-cloud architecture.

Some applications, such as energy (electricity) usage monitoring of smart meters, require periodic data exchange. In this applications, electricity consumption values (active and reactive electricity) are transmitted at regular intervals (every hour) between IoT-enabled smart meters and the cloud-based metering data management system (MDMS) over the Internet as depicted in Fig. 4. Lightweight RESTful APIs (JSON APIs) are deployed to facilitate device-to-device communication and enable periodic metering data exchange using HTTP methods (web services) with minimal effort in the system architecture. These real-time metering data is permanently stored in a data application on the MDMS server, which further utilized this data for billing and energy forecasting in the smart grid. In addition to periodic data exchange, smart meters generate both normal and time-critical data, which requires different operational behaviors and QoS provisioning, such as extensive real-time processing, memory management, optimized bandwidth, low latency, and SLA compliance constraints in the communication network. Fog computing plays a crucial role by providing locally low-latency processing, load balancing, fault tolerance, energy efficiency, while cloud computing offers on demand centralized data storage, computation, and deeper data analytics in large-scale heterogeneous IoT networks with complex QoS requirements. However, these heterogeneous IoT devices generate a massive amount of IoT data with different priorities and traffic rates, which means that appropriate resource allocation, management, and execution of all tasks on fog and cloud nodes become critical in the system. If not properly handled, this may lead to important data loss and delays in completing time-critical applications. The proposed EcoTaskSched model aims to provide an energy-efficient task scheduling approach to address this challenge. The main focus of this study is the scheduling of tasks  $\{t_1, t_2, \dots, t_N\}$  on fog nodes  $\{F_1, F_2, \dots, F_N\}$  or cloud nodes  $\{C_1, C_2, \dots, C_N\}$ , ensuring that overall energy consumption is minimized while maintaining QoS in the fog-cloud environment. The research problem is formulated as an objective function (expressed in Eq. 1), and the solution is implemented in Algorithm 1.

**Performance evaluation**

In this section, we evaluate the proposed EcoTaskSched model as detailed in the previous section. We conduct extensive experiments under different workload scenarios to validate its effectiveness in comparison to the base line models: GGCN<sup>25</sup> and BiGGCN<sup>26</sup>. First we discuss the experimental setup and evaluation metrics, followed by detailed analysis of the experimental results in Sect. 7.

Experimental setup

This section details the experimental setup for evaluating the proposed EcoTaskSched model in comparison to the baseline models in a fog-cloud computing environment. We conduct experiments using the COSCO framework, a physical testbed enabling single or multi-step simulations of container migration decisions in fog computing. To simulate real deployments, we setup a physical setup of 10 host VMs on the Microsoft Azure general purpose Bs-series to create heterogeneous fog-cloud testbed for large-scale experiments. Specifically, we use six lightweight VMs in the fog (edge) layer, hosted in the West UK datacenter, and four powerful VMs located in the US Central region, running on the Windows OS platform in the fog and cloud layer. Bs-series VMs offer a cost-effective option for workloads with low to moderate CPU usage, making them ideal for testing, development, low to medium-traffic web servers, and small to medium databases. We assume that all gateway connectivity devices are on the same LAN as the host server in the West UK region. Table 5 presents host configurations and pricing details (cost per hour in US Dollar) at the fog and cloud layer.

Similarly, the Defog benchmark is a suit of six real-time heterogeneous workloads used to generate workloads for training and evaluating the performance of the EcoTaskSched model against baseline models in fog computing. In a smart building, certain DeFog workloads are particularly relevant to heterogeneous IoT applications, such as video surveillance processing, environmental monitoring, and smart lighting control. We utilize three specific DeFog applications: the Yolo benchmark (compute-intensive, memory-intensive, and bandwidth-intensive), Aeneas benchmark (bandwidth-intensive), and PocketSphinx benchmark (compute-intensive), which run workloads corresponding to a variety of heterogeneous IoT applications in a fog-cloud environment. Yolo and Aeneas runs with 1500 user requests, whereas PocketSphinx runs with 320 user requests. We use these different categories of workloads to generate training data and run our simulations. The dataset consists of workload information on resource utilization metrics for each time step, including the number of CPU cores requested, CPU frequency (MIPS), Disk read/write operations, RAM requested, and network bandwidth (receive/transmit) characteristics from VMs running on distributed datacenters. These recourse characteristics and workload data constitutes the features values set. Furthermore, the arrival of all independent tasks within a time period (scheduling interval ( $I_t$ )), follows a Poisson distribution, i.e., Poisson ( $\lambda$ ) where  $\lambda = 1.2$  jobs for  $|H|=10$  and  $\lambda = 5$  jobs for  $|H|=50$ . Here, H represents the number of virtualized host, with 10 hosts in the Azure physical setup and 50 hosts considered in a large-scale fog environment in the simulator. The task distribution and selection among the three DeFog applications (Yolo, Aeneas, and PocketSphinx) follow the probabilities ( $p_y, p_a, p_s$ ) = 0.33 for each run. We conduct all experiments over 50 scheduling interval, where each interval lasts 300 s. A total experiment time of 4 h and 10 min was consumed. To ensure statistical significance, we the results over five runs and use diverse workload scenarios (normal and intensive) to compare with the baseline models. We implemented the proposed algorithm and baseline models in Python (TensorFlow) using the COSCO framework and the DeFog benchmark, integrating CNN and BiLSTM models in a fog-cloud environment. All experiments were conducted on a computer system with the following hardware specifications: CPU (Intel i7-10700 K), RAM (64GB), GPU (Nvidia GTX 1060) and OS (Windows 10 Pro).

Evaluation metrics

We comprehensively evaluate the EcoTaskSched model’s performance using key metrics that assess its effectiveness in task scheduling and resource management within fog-cloud computing environments. The primary metrics include energy consumption, job completion rate, SLA violation rate, response time, cost time, and throughput. Each metric is detailed below, along with its mathematical formulation.

Energy consumption

Energy consumption is a critical metric as it directly impacts the operational costs and environmental footprint of cloud data centers. The total energy consumption per completed task is calculated in kilowatt-hours (kWh), and is formulated in Eq. (19).

$$E_{Total} = \sum_{t=1}^N E_{Consumption}$$
 (19)

Here,  $E_{Consumption}$  represents the total energy consumed by computing resources (e.g., CPU, HDD, RAM, network, and additional devices such as the motherboard, connectors, and cooling fans) during job processing and completion, as expressed in Eq. (20).

$$E_{Consumption} = E_{CPU} + E_{HDD} + E_{RAM} + E_{Network} + E_{Additional}$$
 (20)

| Instance | Quantity | Core (s) | MIPS | RAM (GiB) | Temporary storage (GiB) | Network Bandwidth | Ping time | Cost Model (Pay as you go with AHB) |
|----------|----------|----------|------|-----------|-------------------------|-------------------|-----------|-------------------------------------|
| B2s      | 4        | 2        | 4029 | 4         | 8                       | 1000 MB/s         | 3 ms      | \$0.0470/h                          |
| B4ms     | 2        | 4        | 8102 | 16        | 32                      | 1000 MB/s         | 3 ms      | \$0.1880/h                          |
| B4ms     | 2        | 4        | 8102 | 16        | 32                      | 1000 MB/s         | 76 ms     | \$0.2000/h                          |
| B8ms     | 2        | 8        | 2000 | 32        | 64                      | 2500 MB/s         | 76 ms     | \$0.3990/h                          |

Table 5. Microsoft Azure VMs host capacities and pricing details<sup>37</sup>.

For example,  $E_{CPU}$  is calculated using Eq. (21) as follows:

$$E_{CPU} = \sum_{p=1}^{core} (E_{p_{dynamic}} + E_{p_{idle}}) \quad (21)$$

Here,  $E_{p_{dynamic}}$  represents the dynamic energy consumption, and  $E_{p_{idle}}$  represents the idle energy consumption of the  $p^{th}$  core.

#### Job completion rate

The job completion rate measures the efficiency of the task scheduling process by comparing the total number of completed jobs ( $\sum J_{completed}$ ) to the total assigned jobs ( $\sum J_{assigned}$ ), as expressed in Eq. (22).

$$Job\ Completion\ Rate = \frac{\sum J_{completed}}{\sum J_{assigned}} \times 100 \quad (22)$$

#### SLA violation rate

SLA violations indicate how often the system fails to meet the agreed-upon performance standards. The SLA violation rate is calculated as the ratio of violated SLAs to the total jobs, as expressed in Eq. (23).

$$SLA\ Violation\ Rate = \frac{\sum SLA_{violation}}{\sum J_{total}} \times 100 \quad (23)$$

#### Response time

Response time measures how quickly the system completes a task. It is calculated as the average time from job submission to completion, as shown in Eq. (24).

$$Average\ Response\ Time = \frac{\sum T_{response}}{N} \quad (24)$$

#### VM execution (Compute) time cost

The cost for executing a workflow (task) in Azure VM is calculated in Eq. (25):

$$Cost_{compute} = \sum_{i=1}^N \left( \frac{T_i}{60} \times P_i \right) \quad (25)$$

where  $T_i = (Finish\ Time - Start\ Time)_i$  represents the execution time (in minutes) of task  $i$ ,  $P_i$  is the price per hour of running task  $i$  on the selected Azure VM instance, and  $N$  denotes the total number of tasks in a workflow.

#### Throughput

Throughput measures the total number of jobs completed ( $\sum J_{completed}$ ) in per unit time (e.g., per second), expressed in Eq. (26):

$$Throughput = \frac{\sum J_{completed}}{Totaltime} \quad (26)$$

The next section presents and discusses the experimental results of the EcoTaskSched model, calculated using these evaluation metrics, demonstrating that the objective function effectively addresses the research problem.

## Results and discussion

In this section, we analyze and evaluate the results obtained from our proposed EcoTaskSched model using all the specified performance evaluation metrics. The primary objective of the EcoTaskSched model is to optimize task scheduling on the constrained resources of fog and cloud to reduce energy consumption, as expressed in Eq. (1) (the objective function), while ensuring QoS provisioning, as described in Eqs. (1a)-(1d), using a CNN-based model coupled with BiLSTM network. The experimental results of the EcoTaskSched model are quantified, visualized, and compared with baseline models, including the traditional GGCN<sup>25</sup> and BiGGCN<sup>26</sup>. The detailed analysis is based on experiments conducted under various workload scenarios using the COSCO framework, a Python-based statistical library. The main advantage of using the COSCO library is its seamless integration of ML-based models for task scheduling in realistic fog-cloud environments. It supports simulations of complex scenarios and facilitates benchmarking and evaluation. Two types of scenarios were tested: a normal workload (with a mean of 2 tasks per interval) and an intensive workload (with a mean of 4 tasks per interval). Each scenario was tested on four standard Azure B2s virtualized host machines. Each experiment ran for 50 scheduling intervals, with tasks generated and normally distributed using the DeFog benchmark.



### Energy consumption

First, we present and analyze the energy consumption per completed task for both normal workload and intensive workload conditions, for the EcoTaskSched model in comparison to the GGCN<sup>25</sup> and BiGGCN<sup>26</sup> models, as shown in Figs. 5 and 6, respectively.

As shown in Table 6, the EcoTaskSched model significantly outperforms both GGCN<sup>25</sup> and BiGGCN<sup>26</sup> in terms of energy consumption per completed task under normal workload conditions. The average energy consumption per task for EcoTaskSched is 65.8 kWh, which is notably lower than 109.3 kWh for GGCN<sup>25</sup> and 95.1 kWh for BiGGCN<sup>26</sup>.

In contrast, under intensive workload conditions, as shown in Table 7, EcoTaskSched model maintains its efficiency, with an average energy consumption of 68.9 kWh per task.

This is significantly lower than that of GGCN (98.4 kWh) and BiGGCN (83.8 kWh), as illustrated in Fig. 6. The experimental results clearly demonstrate that the EcoTaskSched model effectively reduces energy consumption for task scheduling across various resources, including CPU, RAM, Disk\_IO, and Network\_IO in fog-cloud computing environments. Its advanced architecture, integrating CNN and BiLSTM techniques, optimizes resource utilization, resulting in significant energy savings.

### Job completion rate

The job completion rate is a crucial metric for assessing the effectiveness of the EcoTaskSched model in managing and scheduling tasks within a fog-cloud computing environment. It measures the proportion of successfully completed tasks out of the total assigned, reflecting the model's efficiency in handling workloads and meeting performance requirements. As shown in Table 8, the EcoTaskSched model achieves a significantly higher job completion rate under normal workload conditions compared to the GGCN and BiGGCN models.

The average job completion rate for EcoTaskSched is 85%, while GGCN and BiGGCN achieve 64% and 73%, as depicted in Fig. 7.

In contrast, under intensive workload conditions, as detailed in Table 9, the EcoTaskSched model maintains its superior performance with an average job completion rate of 84.5%, which is considerably higher than that of GGCN (69%) and BiGGCN (77%), respectively.

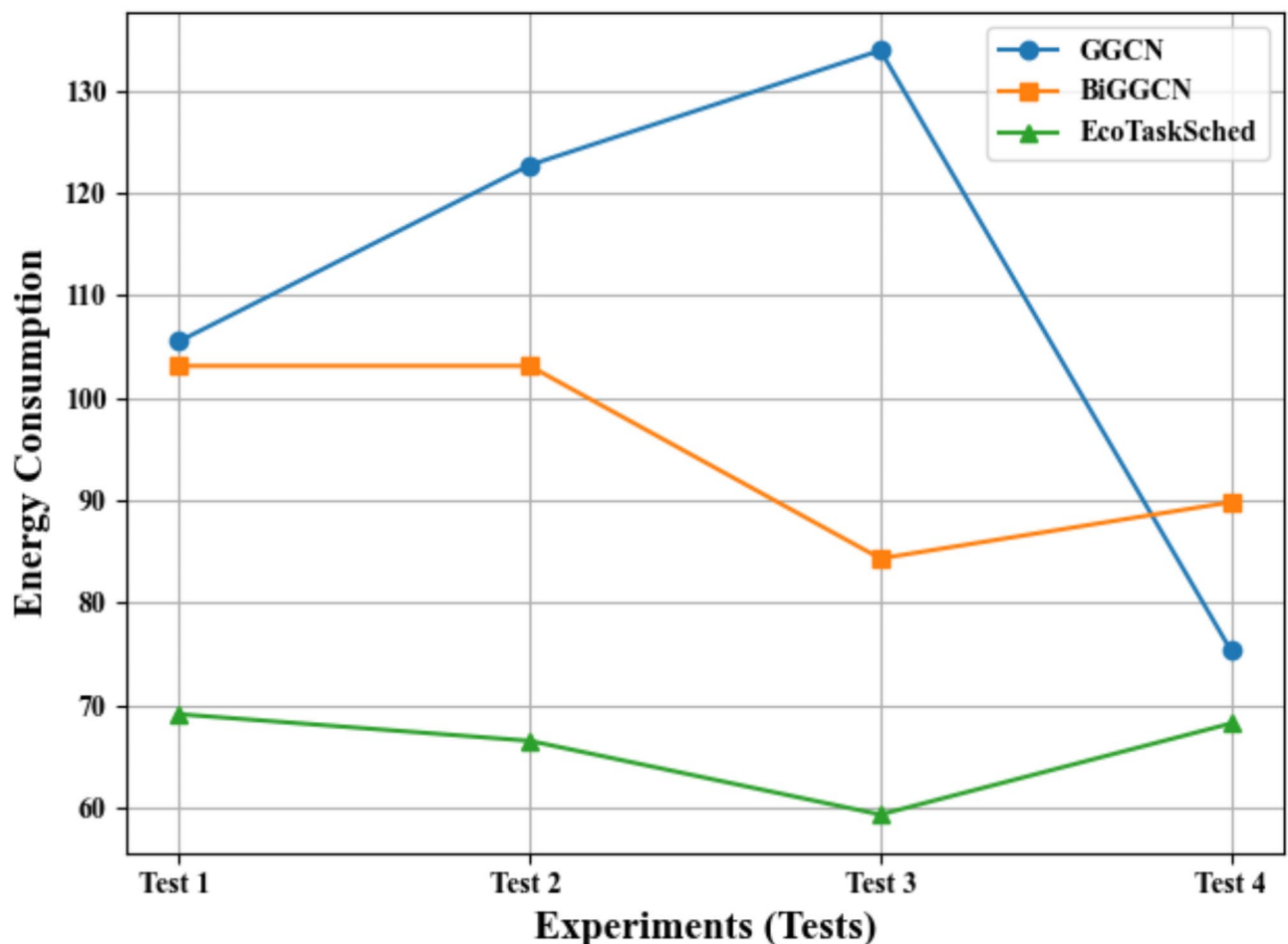
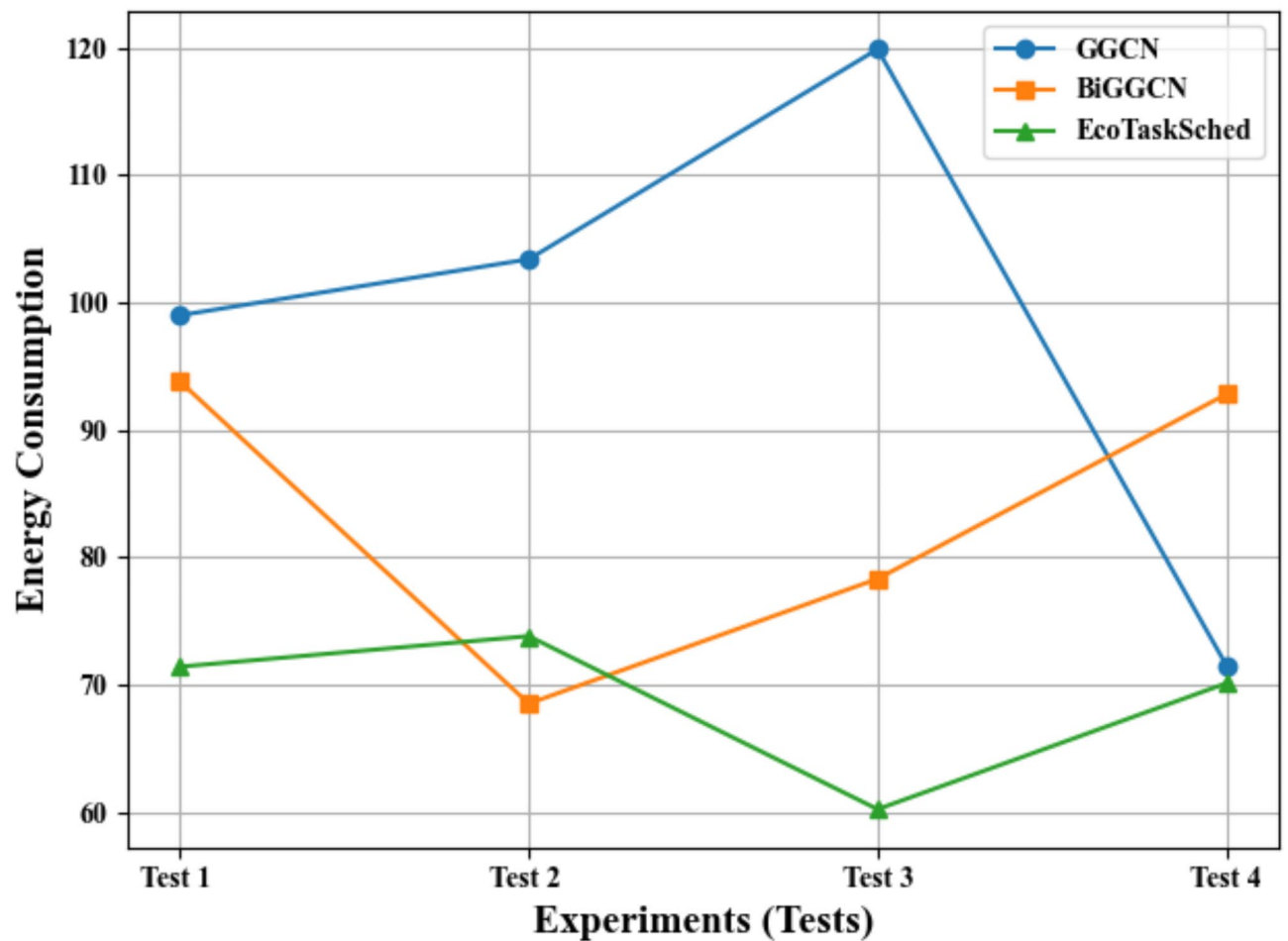


Fig. 5. Energy consumption per completed task under a normal workload.



**Fig. 6.** Energy consumption per completed task under an intensive workload.

| Model                | Test 1 | Test 2 | Test 3 | Test 4 | Average |
|----------------------|--------|--------|--------|--------|---------|
| GGCN <sup>25</sup>   | 105.5  | 122.7  | 133.9  | 75.3   | 109.3   |
| BiGGCN <sup>26</sup> | 103.1  | 103.1  | 84.3   | 89.8   | 95.1    |
| EcoTaskSched         | 69.1   | 66.5   | 59.3   | 68.2   | 65.8    |

**Table 6.** Comparison of energy consumption for normal workloads.

| Model                | Test 1 | Test 2 | Test 3 | Test 4 | Average |
|----------------------|--------|--------|--------|--------|---------|
| GGCN <sup>25</sup>   | 99.0   | 103.4  | 119.9  | 71.5   | 98.4    |
| BiGGCN <sup>26</sup> | 93.8   | 68.5   | 78.3   | 92.8   | 83.8    |
| EcoTaskSched         | 71.4   | 73.8   | 60.2   | 70.1   | 68.9    |

**Table 7.** Comparison of energy consumption for intensive workloads.

| Model                | Test 1 | Test 2 | Test 3 | Test 4 | Average |
|----------------------|--------|--------|--------|--------|---------|
| GGCN <sup>25</sup>   | 68     | 51     | 51     | 84     | 64      |
| BiGGCN <sup>26</sup> | 71     | 66     | 81     | 76     | 73      |
| EcoTaskSched         | 86     | 86     | 90     | 77     | 85      |

**Table 8.** Comparison of job completion rate for normal workloads (%).

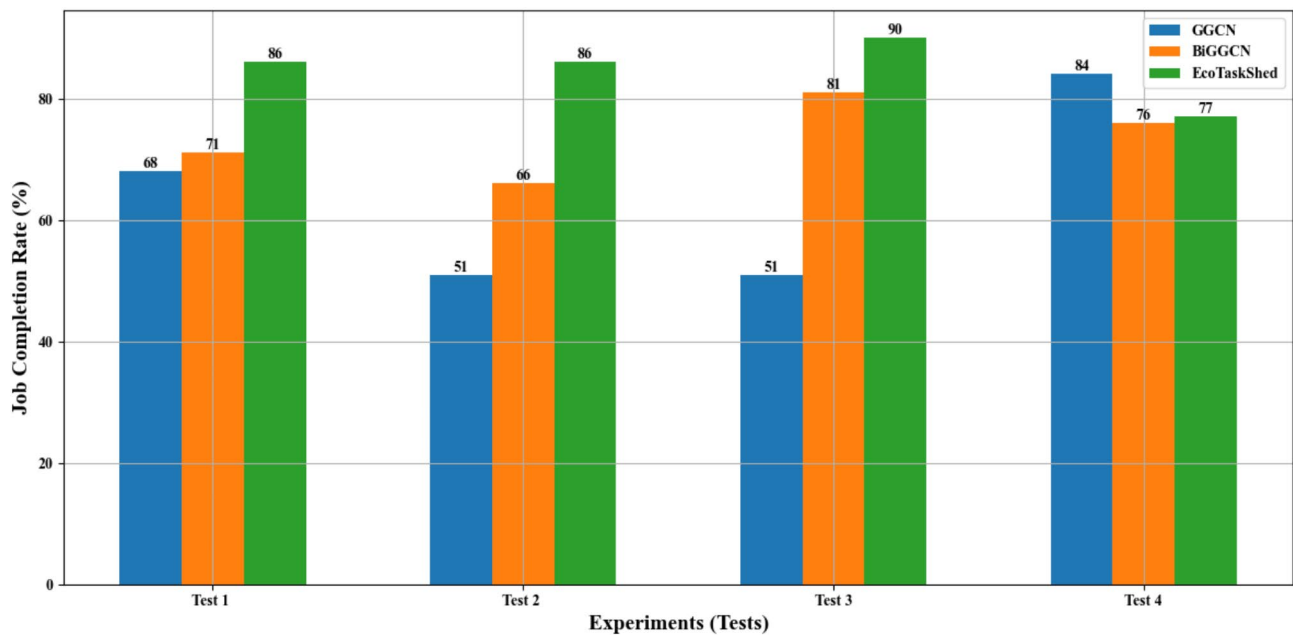


Fig. 7. Comparison of job completion rate for normal workloads (%).

| Model                | Test 1 | Test 2 | Test 3 | Test 4 | Average |
|----------------------|--------|--------|--------|--------|---------|
| GGCN <sup>25</sup>   | 64     | 65     | 61     | 85     | 69      |
| BiGGCN <sup>26</sup> | 68     | 88     | 72     | 79     | 77      |
| EcoTaskSched         | 81     | 89     | 91     | 77     | 84.5    |

Table 9. Comparison of job completion rate for intensive workloads (%).

The experimental results depicted in Fig. 8 indicate that the EcoTaskSched model excels at completing assigned tasks, making it highly effective for task scheduling in fog-cloud computing environments. Its advanced architecture, which integrates a CNN with a BiLSTM network, enables it to handle dynamic workloads efficiently, ensuring a higher proportion of tasks are completed successfully.

Task allocation distribution

Additionally, understanding the task allocation behavior of the EcoTaskSched model provides insights into its efficiency in assigning tasks to available resources, ensuring optimal utilization while minimizing latency and energy consumption. This section analyzes the task allocation patterns of the EcoTaskSched model compared to the baseline models (GGCN and BiGGCN) under different workload conditions. As shown in Table 10, the EcoTaskSched model achieves a balanced task allocation across all hosts under normal workload conditions, with each host receiving an equal number of tasks (25 each). This uniform distribution demonstrates the proposed model’s efficiency in evenly distributing the load, preventing any single host from becoming a bottleneck. In contrast, GGCN and BiGGCN exhibit greater variability in task allocation, resulting in uneven load distribution.

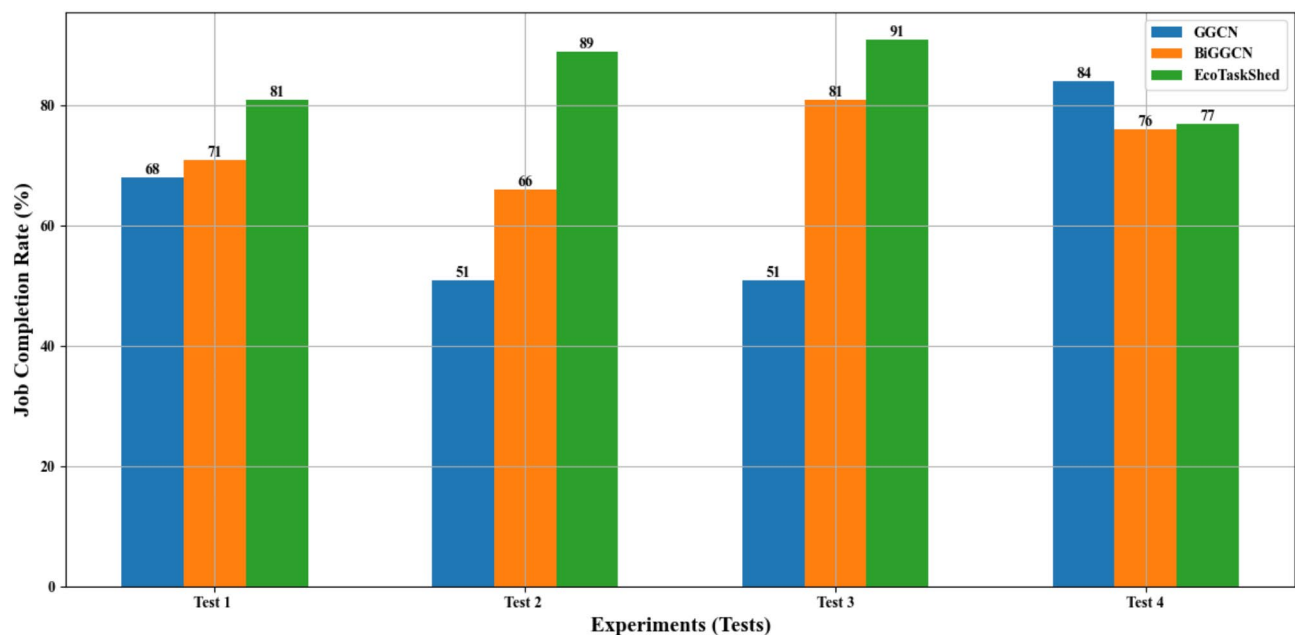
In contrast, under intensive workload conditions, as shown in Table 11, the EcoTaskSched model maintains its balanced task allocation behavior, with each host receiving 50 tasks. This consistency demonstrates the model’s robustness in handling higher workload scenarios. However, GGCN and BiGGCN exhibit greater variability in task distribution, which could lead to resource contention and inefficiencies.

Figure 9 illustrates the task allocation distribution across four hosts for the GGCN, BiGGCN, and the EcoTaskSched models. Each subplot shows the distribution of tasks over different intervals, highlighting how effectively each model balances the workload among the virtualized hosts.

The task allocation distribution behavior of the EcoTaskSched model highlights its capability to distribute tasks uniformly across available resources, ensuring balanced load distribution and efficient resource utilization. This behavior contributes to the model’s superior performance in minimizing energy consumption and increasing job completion rates.

SLA violation rate and average response time

To demonstrate the effectiveness of the EcoTaskSched model, it is essential to compare its performance with baseline models (GGCN<sup>25</sup> and BiGGCN<sup>26</sup>), focusing on key metrics such as SLA violation rates and average response time. As shown in Table 12, the EcoTaskSched model achieves a low SLA violation rate of 15%, demonstrating its effectiveness in meeting SLAs and maintaining QoS requirements. In comparison, GGCN and



**Fig. 8.** Comparison of job completion rate for intensive workloads (%).

| Model                | Host 1 | Host 2 | Host 3 | Host 4 | Total Tasks |
|----------------------|--------|--------|--------|--------|-------------|
| GGCN <sup>25</sup>   | 20     | 25     | 30     | 25     | 100         |
| BiGGCN <sup>26</sup> | 23     | 27     | 25     | 25     | 100         |
| EcoTaskSched         | 25     | 25     | 25     | 25     | 100         |

**Table 10.** Comparison of task allocation distribution for normal workloads.

| Model                | Host 1 | Host 2 | Host 3 | Host 4 | Total Tasks |
|----------------------|--------|--------|--------|--------|-------------|
| GGCN <sup>25</sup>   | 45     | 55     | 60     | 40     | 200         |
| BiGGCN <sup>26</sup> | 48     | 52     | 50     | 50     | 200         |
| EcoTaskSched         | 50     | 50     | 50     | 50     | 200         |

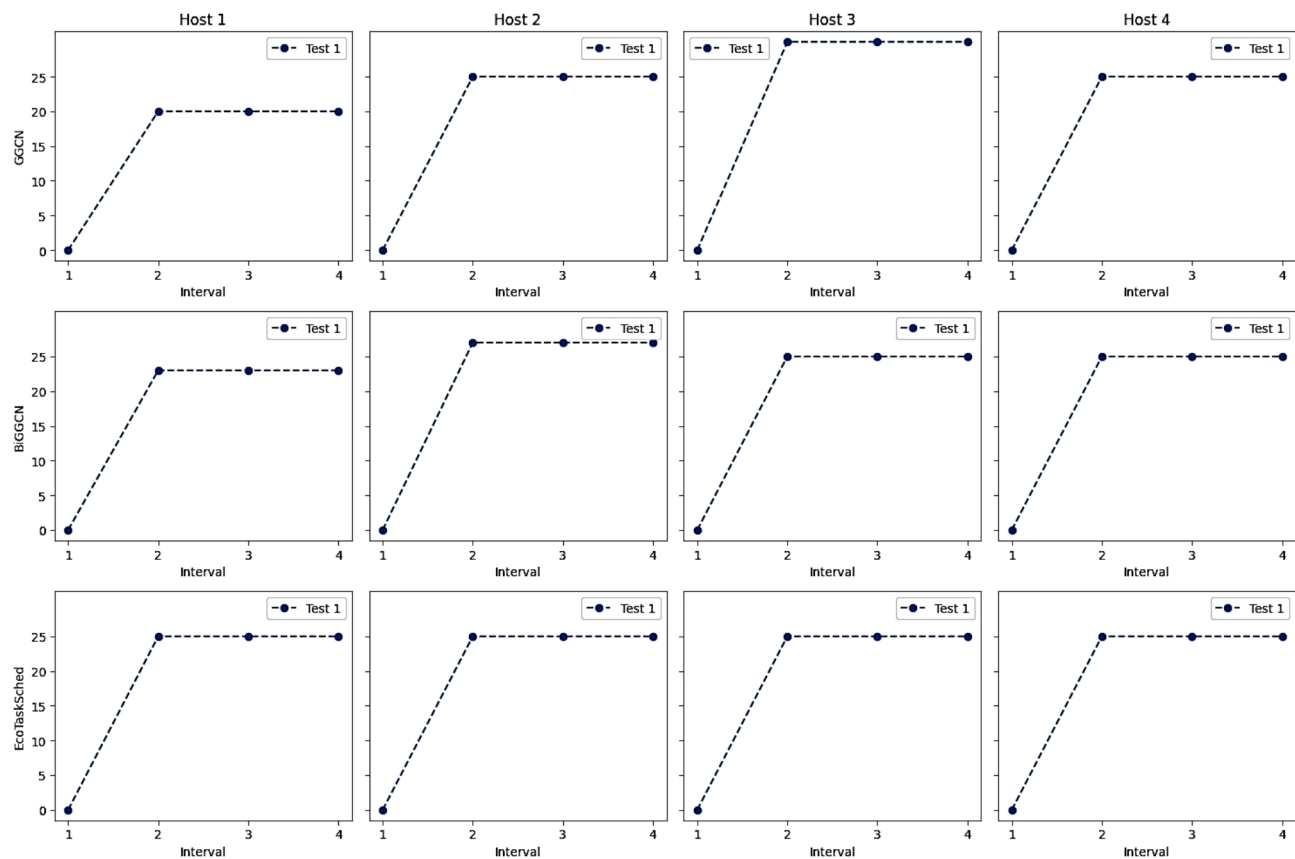
**Table 11.** Comparison of task allocation distribution for intensive workloads.

BiGGCN have higher SLA violation rates of 33.5% and 25%, respectively. Similarly, the EcoTaskSched model has the shortest average response time of 2.5 s, compared to GGCN (4.35 s) and BiGGCN (3.9 s). This reduction in average response time is crucial for real-time IoT applications and enhances the overall service users experience.

### VM execution (compute) time cost

In Table 13, we take into account only the processing cost (refer to Table 5) and analyze the cost of our proposed EcoTaskSched model in comparison to baseline models (GGCN<sup>25</sup> and BiGGCN<sup>26</sup>) using different Microsoft Azure virtualized hosts (B2s, B4ms, B8ms) plans under normal and intensive workload scenarios.

For example, in the normal workload conditions, the GGCN<sup>25</sup> model incurs a higher average execution cost (\$11.55), while the BiGGCN<sup>26</sup> model has a slightly higher costs (\$11.86). In contrast, the proposed EcoTaskSched model incurs a significantly lower execution cost (\$7.35). Our proposed model reduces the execution cost by 36% compared to the baseline models, making it more efficient in normal workload scenarios. Similarly, in intensive workloads, the GGCN<sup>25</sup> model incurs the highest execution cost (\$12.27), while the BiGGCN<sup>26</sup> model has a lower cost (\$9.16), performing better than the GGCN. In contrast, the EcoTaskSched model achieves the lowest execution cost (\$7.97), reducing costs by approximately 35% compared to GGCN and outperforming both the baseline models in intensive workloads. In conclusion, the proposed EcoTaskSched model consistently achieves the lowest execution cost in both normal and intensive workload conditions, making it cost-efficient solution task scheduling. Figure 10 depicts the execution time costs both for normal and intensive workloads of the three models (GGCN, BiGGCN, and EcoTaskSched).



**Fig. 9.** Task allocation distribution on hosts for GGCN, BiGGCN, and EcoTaskSched model.

| Model                | SLA violation Rate (%) | Average response time (s) | Performance | Robustness | Workload adaptability |
|----------------------|------------------------|---------------------------|-------------|------------|-----------------------|
| GGCN <sup>25</sup>   | 33.5 (High)            | 4.35 (High)               | Moderate    | Moderate   | Moderate              |
| BiGGCN <sup>26</sup> | 25 (Moderate)          | 3.9 (Moderate)            | Moderate    | High       | High                  |
| EcoTaskSched         | 15 (Low)               | 2.5 (Low)                 | High        | High       | High                  |

**Table 12.** Performance comparison of the GGCN, BiGGCN, and ecotasksched models.

| Workload scenario | Model                | Microsoft Azure Virtualized Host CPU computation (MIPS) vs. price per hour (\$) |          |          |         |          |
|-------------------|----------------------|---------------------------------------------------------------------------------|----------|----------|---------|----------|
|                   |                      | B2s                                                                             | B4ms     | B4ms     | B8ms    | Average  |
| Normal            | GGCN <sup>25</sup>   | 0.6204                                                                          | 2.741667 | 2.7      | 5.4929  | 11.55497 |
|                   | BiGGCN <sup>26</sup> | 0.643117                                                                        | 2.675867 | 2.813333 | 5.72565 | 11.85797 |
|                   | EcoTaskSched         | 0.377567                                                                        | 1.576067 | 1.79     | 3.6043  | 7.347933 |
| Intensive         | GGCN <sup>25</sup>   | 0.6627                                                                          | 2.729133 | 2.946667 | 5.9318  | 12.2703  |
|                   | BiGGCN <sup>26</sup> | 0.480967                                                                        | 1.9646   | 2.13     | 4.58185 | 9.157417 |
|                   | EcoTaskSched         | 0.409683                                                                        | 1.726467 | 1.89     | 3.94345 | 7.9696   |

**Table 13.** Execution time cost.

### Throughput

Throughput is an important characteristic for evaluating the performance of the EcoTaskSched model. According to Table 14, the proposed EcoTaskSched model achieved the highest average throughput across all configurations of Azure virtualized hosts compared to the other two models. Due to its excellent scheduling



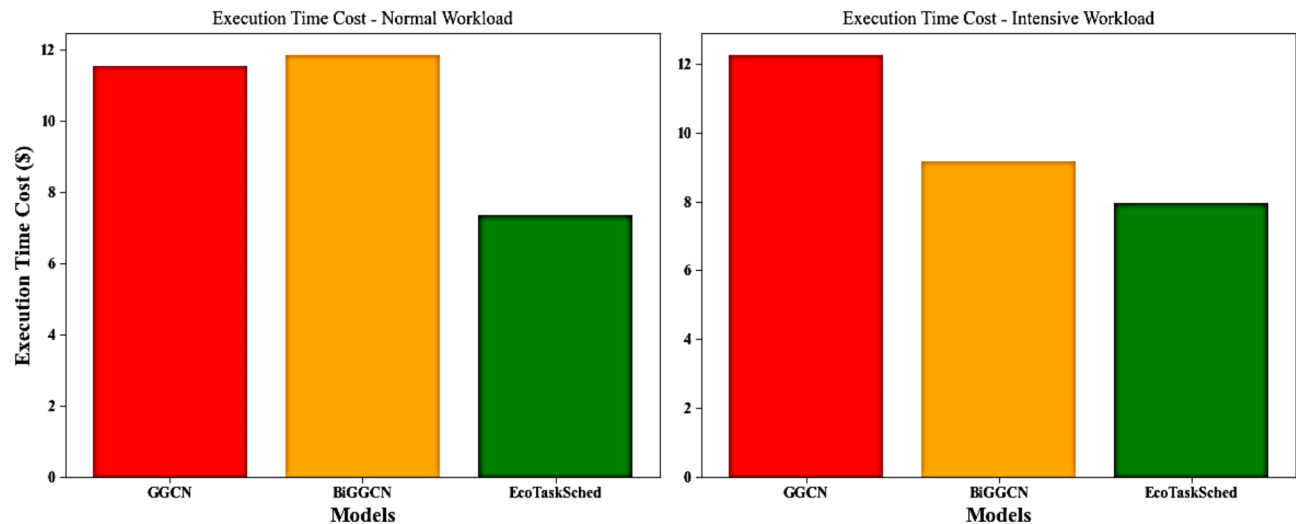


Fig. 10. Execution time cost comparison for normal and intensive workloads.

| Model                | Throughput (tasks/second) | Scheduling efficiency | Energy edfficiency | Scalability   |
|----------------------|---------------------------|-----------------------|--------------------|---------------|
| GGCN <sup>25</sup>   | 12.3                      | Good                  | Moderate           | Moderate      |
| BiGGCN <sup>26</sup> | 13.5                      | Very good             | Moderate-high      | Moderate-high |
| EcoTaskSched         | 15.8                      | Excellent             | High               | High          |

Table 14. Throughput comparison of models.

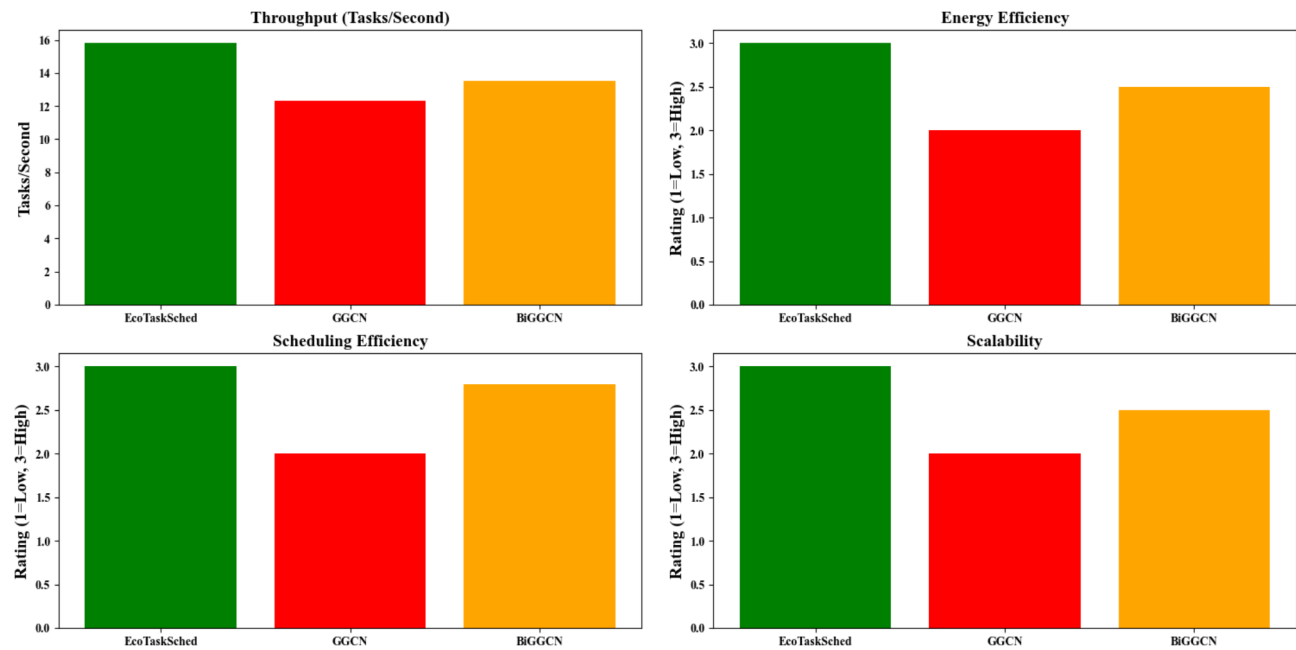


Fig. 11. Throughput comparison of models.

strategy, the proposed model is energy-efficient in task allocation, effective in load balancing and highly scalable, demonstrating its superiority in handling tasks in large-scale IoT-based fog-cloud systems as shown in Fig. 11.

The EcoTaskSched model outperforms the baseline models across all key performance metrics. It achieves lower energy consumption, higher job completion rates, fewer SLA violations, shorter average response times, lower execution cost, and improved throughput. These results validate the effectiveness of the EcoTaskSched

model in optimizing task scheduling and resource management in an energy-efficient manner within fog-cloud computing environments.

## Conclusion and future work

The integration of IoT technologies into fog-cloud computing networks provides practical solutions and adds multiple dimensions to the modern digital landscape. Cloud data centers deliver real-time data, and insights to service users, enabling monitoring, control, productivity improvements, and decision-making. This advancement also promotes environmental sustainability. However, this extensive traffic exchange raises concerns about the uniform allocation and efficient utilization of constrained resources (software, hardware, and third-party products) to ensure QoS parameters such as latency, energy consumption, job completion ratio, SLA, etc. in fog-cloud of IoT network. To address these challenges, we defined an objective function to reduce energy consumption while ensuring QoS provisioning in the fog-cloud of IoT network. Therefore, we proposed the EcoTaskSched model, which significantly enhances cloud stream processing by integrating advanced ML techniques, particularly CNN with BiLSTM network, to enable more efficient task scheduling. Meanwhile, the stable CNN model is employed to identify and extract feature sets from time-series workload tasks and constrained resources of the fog and cloud nodes. BiLSTM is integrated with the convolutional layers of the CNN model to capture both past and future dependencies in the input sequence. This enable efficient sequential decision-making in task scheduling and helps resolve the research problem. The research problem is mathematically expressed through an objective function, and a scheduling algorithm is developed to support the proposed methodology. This approach outperforms state-of-the-art models by reducing energy consumption, increasing job completion rates, minimizing SLA violations, lowering response rates, improving throughput, and reducing execution costs in a dynamic and heterogeneous fog-cloud IoT environment. Furthermore, the experimental results show that the EcoTaskSched model achieves an average energy consumption of 65.8 kWh under normal workloads and 68.9 kWh under intensive conditions, significantly lower than that of the GGCN and BiGGCN models. Additionally, the EcoTaskSched model consistently achieved a high average job completion rate of 85% under normal workloads and 84.5% intensive workload conditions, surpassing GGCN (64% and 69%) and BiGGCN (73% and 77%). Similarly, the EcoTaskSched model has lower SLA violation rates and average response times compared to the other baseline models. The EcoTaskSched model reduces execution costs by 36% and 35%, outperforming the baseline models. Furthermore, it achieves the highest average throughput across all configurations of virtualized hosts compared to the other two models. Based on these significant improvements in the required parameters achieved by the proposed model, we conclude that the objective function effectively addresses the research problem in the fog-cloud IoT network.

In future research, we will focus on further validating the EcoTaskSched model through real-world IoT applications in cloud and fog environments of varying scales. Also, investigating the model's scalability and robustness across diverse conditions and operations will be essential. Additionally, integrating AI models with other advanced ML techniques could further enhance business process design.

## Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Received: 6 January 2025; Accepted: 1 April 2025

Published online: 10 April 2025

## References

1. Batoool, R. et al. Green is clean: the role of ICT in resource management. *Environ. Sci. Pollut. Res.* **26**(24), 25341–25358. <https://doi.org/10.1007/s11356-019-05748-0> (2019).
2. Islam, R. et al. The future of cloud computing: Benefits and challenges. *Int. J. Commun. Netw. Syst. Sci.* **16**(04), 53–65. <https://doi.org/10.4236/ijcns.2023.164004> (2023).
3. Yenugula, M., Sahoo, S. K. & Goswami, S. S. Cloud computing in supply chain management: Exploring the relationship. *Manag. Sci. Lett.* **13**(3), 193–210. <https://doi.org/10.5267/j.msl.2023.4.003> (2023).
4. Deepak, M., Upadhyay, K., Alam, M. Load balancing techniques in fog and edge computing: issues and challenges. In *2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT)*, vol 5, 210–215. <https://doi.org/10.1109/IC2PCT60090.2024.10486765>, (2024).
5. Ali, H., Abidin, S., Alam, M. Auditing of outsourced data in cloud computing: An overview. In *2024 11th International Conference on Computing for Sustainable Global Development (INDIACom)*, 111–117. <https://doi.org/10.23919/INDIACom61295.2024.10498177>, (2024).
6. Alam, M., Mustajab, S., Shahid, M., Ahmad, F. Cloud computing: Architecture, vision, challenges, opportunities, and emerging trends. In *2023 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, 829–834. <https://doi.org/10.1109/ICCCIS60361.2023.10425507> (2023).
7. Srirama, S. N. A decade of research in fog computing: Relevance, challenges, and future directions. *Softw. Pract. Exp.* **54**(1), 3–23. <https://doi.org/10.1002/spe.3243> (2024).
8. Huaranga-Junco, E., González-Gerpe, S., Castillo-Cara, M., Cimmino, A. & García-Castro, R. From cloud and fog computing to federated-fog computing: A comparative analysis of computational resources in real-time IoT applications based on semantic interoperability. *Futur. Gener. Comput. Syst.* **159**(October 2023), 134–150. <https://doi.org/10.1016/j.future.2024.05.001> (2024).
9. Habib, F. et al. Deep neural networks for enhanced security: Detecting metamorphic malware in IoT devices. *IEEE Access* **12**, 48570–48582. <https://doi.org/10.1109/ACCESS.2024.3383831> (2024).
10. Schafhalter, P., Kalra, S., Xu, L., Gonzalez, J. E., Stoica, I. Leveraging cloud computing to make autonomous vehicles safer. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5559–5566. <https://doi.org/10.1109/IROS55552.2023.10341821> (2023).
11. Khan, A. et al. A QoS-aware machine learning-based framework for AMI applications in smart grids. *Energies* **14**(23), 8171 (2021).

12. Alulfareh, M. F., Humayun, M., Ahmad, Z. & Khan, A. An intelligent LoRaWAN-based IoT device for monitoring and control solutions in smart farming through anomaly detection integrated with unsupervised machine learning. *IEEE Access* **12**, 119072–119086. <https://doi.org/10.1109/ACCESS.2024.3450587> (2024).
13. Janeera, D. A., Gnanamalar, S. S. R., Ramya, K. C. & Kumar, A. G. A. Internet of Things and artificial intelligence-enabled secure autonomous vehicles for smart cities BT—Automotive embedded systems: Key technologies, innovations, and applications. In *Automotive Embedded Systems: Key Technologies, Innovations, and Applications* (eds Kathiresh, M. & Neelaveni, R.) 201–218 (Springer, 2021). [https://doi.org/10.1007/978-3-030-59897-6\\_11](https://doi.org/10.1007/978-3-030-59897-6_11).
14. Khan, M. S. et al. Improved multi-model classification technique for sound event detection in urban environments. *Appl. Sci.* **12**(19), 9907. <https://doi.org/10.3390/app12199907> (2022).
15. Qazi, S. et al. m5C-TNKmer: Identification of 5-methylated base cytosine of ribonucleic acid using supervised machine learning techniques. *Eng. Rep.* <https://doi.org/10.1002/eng2.13073> (2024).
16. Shahzad, M. et al. Identification of anemia and its severity level in a peripheral blood smear using 3-tier deep neural network. *Appl. Sci.* **12**(10), 5030. <https://doi.org/10.3390/app12105030> (2022).
17. Shah, D., Asmat, M., Khan, U., Abrar, M. & Tahir, M. Optimizing breast cancer detection with an ensemble deep learning approach. *Int. J. Intell. Syst.* <https://doi.org/10.1155/2024/5564649> (2024).
18. Angel, N. A., Ravindran, D., Vincent, P. M., Srinivasan, K. & Hu, Y.-C. Recent advances in evolving computing paradigms: Cloud, edge, and fog technologies. *Sensors* **22**(1), 196. <https://doi.org/10.3390/s22010196> (2022).
19. Ahmed, E. et al. The role of big data analytics in Internet of Things. *Comput. Netw.* **129**, 459–471. <https://doi.org/10.1016/j.comnet.2017.06.013> (2017).
20. Satya, R. & Dittakavi, S. AI optimized cost-aware design strategies for resource—Efficient applications. *J. Sci. Tech.* **4**(1), 1–10 (2023).
21. Rahimikhanghah, A., Tajkey, M., Rezazadeh, B. & Rahmani, A. M. Resource scheduling methods in cloud and fog computing environments: A systematic literature review. *Cluster Comput.* **25**(2), 911. <https://doi.org/10.1007/s10586-021-03467-1> (2022).
22. Nguyen, B. M., Binh, H. T. T., Anh, T. T. & Son, D. B. Evolutionary algorithms to optimize task scheduling problem for the IoT based bag-of-tasks application in cloud-fog computing environment. *Appl. Sci.* **9**(9), 1730. <https://doi.org/10.3390/app9091730> (2019).
23. Hoseiny, F., Azizi, S., Shojafar, M. & Tafazolli, R. Joint QoS-aware and cost-efficient task scheduling for fog-cloud resources in a volunteer computing system. *ACM Trans. Internet Technol.* **21**(4), 1. <https://doi.org/10.1145/3418501> (2021).
24. Blasch, E. et al. Machine learning/artificial intelligence for sensor data fusion—opportunities and challenges. *IEEE Aerosp. Electron. Syst. Mag.* **36**(7), 80–93. <https://doi.org/10.1109/MAES.2020.3049030> (2021).
25. Tuli, S. et al. HUNTER: AI based holistic resource management for sustainable cloud computing. *J. Syst. Softw.* **184**, 111124. <https://doi.org/10.1016/j.jss.2021.111124> (2022).
26. Ifikhar, S. et al. HunterPlus: AI based energy-efficient task scheduling for cloud–fog computing environments. *Internet of Things* <https://doi.org/10.1016/j.iot.2022.100667> (2023).
27. Khan, A. et al. QoS-aware cost minimization strategy for AMI applications in smart grid using cloud computing. *Sensors* **22**(13), 4969. <https://doi.org/10.3390/s22134969> (2022).
28. Kar, I., Parida, R. N. R., Das, H. Energy aware scheduling using genetic algorithm in cloud data centers. In *Int. Conf. Electr. Electron. Optim. Tech. ICEEOT 2016*, no April, 3545–3550. <https://doi.org/10.1109/ICEEOT.2016.7755364> (2016).
29. Changtian, Y., Jiong, Y. Energy-aware genetic algorithms for task scheduling in cloud computing. In *2012 Seventh ChinaGrid Annual Conference*, 43–48. <https://doi.org/10.1109/ChinaGrid.2012.15> (2012).
30. He, X., Sun, X. & von Laszewski, G. QoS guided min–min heuristic for grid task scheduling. *J. Comput. Sci. Technol.* **18**(4), 442–451. <https://doi.org/10.1007/BF02948918> (2003).
31. Murad, S. S. et al. Optimized min–min task scheduling algorithm for scientific workflows in a cloud environment. *J. Theor. Appl. Inf. Technol.* **100**(2), 480–506 (2022).
32. Abid, A., Manzoor, M. F., Farooq, M. S., Farooq, U. & Hussain, M. Challenges and issues of resource allocation techniques in cloud computing. *KSII Trans. Internet Inf. Syst.* **14**(7), 2815–2839. <https://doi.org/10.3837/tiis.2020.07.005> (2020).
33. Dong, Z., Liu, N. & Rojas-Cessa, R. Greedy scheduling of tasks with time constraints for energy-efficient cloud-computing data centers. *J. Cloud Comput.* <https://doi.org/10.1186/s13677-015-0031-y> (2015).
34. Khan, M. S. A. & Santhosh, R. Task scheduling in cloud computing using hybrid optimization algorithm. *Soft Comput.* **26**(23), 13069–13079. <https://doi.org/10.1007/s00500-021-06488-5> (2022).
35. Khan, A. et al. A QoS-aware data aggregation strategy for resource constrained IoT-enabled AMI network in smart grid. *IEEE Access* **11**(July), 98988–99004. <https://doi.org/10.1109/ACCESS.2023.3312552> (2023).
36. Vignesh, N. et al. Resource management and scheduling in cloud environment. *Int. J. Sci. Res. Publ.* **3**(6), 19–24 (2013).
37. Alam, M., Shahid, M., & Mustajab, S. Security oriented deadline aware workflow allocation strategy for infrastructure as a service clouds. In *2022 3rd International Conference on Computation, Automation and Knowledge Management (ICCAKM)*, 1–6. <https://doi.org/10.1109/ICCAKM54721.2022.9990406> (2022).
38. Witanto, J. N., Lim, H. & Atiquzzaman, M. Adaptive selection of dynamic VM consolidation algorithm using neural network for cloud resource management. *Futur. Gener. Comput. Syst.* **87**, 35–42. <https://doi.org/10.1016/j.future.2018.04.075> (2018).
39. Shanmugam, K., Satyam, K. & Rajasekhar, T. CSO-based efficient resource management for sustainable cloud computing. *Contemp. Math.* **5**(2), 1132–1145. <https://doi.org/10.37256/cm.5220242700> (2024).
40. Tuli, S., Poojara, S. R., Srirama, S. N., Casale, G. & Jennings, N. R. COSCO: Container orchestration using co-simulation and gradient based optimization for fog computing environments. *IEEE Trans. Parallel Distrib. Syst.* **33**(1), 101–116. <https://doi.org/10.1109/TPDS.2021.3087349> (2022).
41. Li, H., Huang, J., Wang, B. & Fan, Y. Weighted double deep Q-network based reinforcement learning for bi-objective multi-workflow scheduling in the cloud. *Cluster Comput.* **25**(2), 751–768. <https://doi.org/10.1007/s10586-021-03454-6> (2022).
42. Ran, Y., Hu, H., Zhou, X., & Wen, Y. DeepEE: Joint optimization of job scheduling and cooling control for data center energy efficiency using deep reinforcement learning. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, 645–655. <https://doi.org/10.1109/ICDCS.2019.00070> (2019).
43. Cao, Z., Wang, R., Zhou, X., & Wen, Y. Toward model-assisted safe reinforcement learning for data center cooling control: A Lyapunov-based approach. In *e-Energy 2023 - Proc. 2023 14th ACM Int. Conf. Futur. Energy Syst.*, 333–346. <https://doi.org/10.1145/3575813.3597343> (2023).
44. Jin, H., Fu, Y., Yang, G. & Zhu, X. An intelligent scheduling algorithm for resource management of cloud platform. *Multimed. Tools Appl.* **79**(7–8), 5335–5353. <https://doi.org/10.1007/s11042-018-6477-4> (2020).
45. Karthik, P. & Sekhar, K. Resource scheduling approach in cloud Testing as a Service using deep reinforcement learning algorithms. *CAAI Trans. Intell. Technol.* **6**(2), 147–154. <https://doi.org/10.1049/cit2.12041> (2021).
46. Bal, P. K., Mohapatra, S. K., Das, T. K., Srinivasan, K. & Hu, Y. C. A joint resource allocation, security with efficient task scheduling in cloud computing using hybrid machine learning techniques. *Sensors* **22**(3), 1242. <https://doi.org/10.3390/s22031242> (2022).
47. Sangpetch, A., Sangpetch, O., Juangmarisakul, N., & Warodom, S. Thoth: Automatic resource management with machine learning for container-based cloud platform. In *CLOSER 2017 - Proc. 7th Int. Conf. Cloud Comput. Serv. Sci.*, Closer, 75–83. <https://doi.org/10.5220/0006254601030111> (2017).
48. Aslani, A. & Ghobaei-Arani, M. Machine learning inference serving models in serverless computing: A survey. *Computing* **107**(1), 47. <https://doi.org/10.1007/s00607-024-01377-9> (2025).

49. Ghorbian, M. & Ghobaei-Arani, M. A survey on the cold start latency approaches in serverless computing: An optimization-based perspective. *Computing* **106**(11), 3755–3809. <https://doi.org/10.1007/s00607-024-01335-5> (2024).
50. Ebrahimi, A., Ghobaei-Arani, M. & Saboohi, H. Cold start latency mitigation mechanisms in serverless computing: Taxonomy, review, and future directions. *J. Syst. Archit.* **151**, 103115. <https://doi.org/10.1016/j.sysarc.2024.103115> (2024).
51. Ghorbian, M., Ghobaei-Arani, M. & Asadolahpour-Karimi, R. Function placement approaches in serverless computing: A survey. *J. Syst. Archit.* **157**, 103291. <https://doi.org/10.1016/j.sysarc.2024.103291> (2024).
52. Aghazadeh, R., Shahidinejad, A. & Ghobaei-Arani, M. Proactive content caching in edge computing environment: A review. *Softw. Pract. Exp.* **53**(3), 811–855. <https://doi.org/10.1002/spe.3033> (2023).
53. Ghorbian, M. & Ghobaei-Arani, M. Function offloading approaches in serverless computing: A survey. *Comput. Electr. Eng.* **120**, 109832. <https://doi.org/10.1016/j.compeleceng.2024.109832> (2024).
54. Jazayeri, F., Shahidinejad, A. & Ghobaei-Arani, M. A latency-aware and energy-efficient computation offloading in mobile fog computing: A hidden Markov model-based approach. *J. Supercomput.* **77**(5), 4887–4916. <https://doi.org/10.1007/s11227-020-03476-8> (2021).
55. Ambigavathi, M. & Sridharan, D. Energy efficient and load balanced priority queue algorithm for wireless body area network. *Futur. Gener. Comput. Syst.* **88**, 586–593. <https://doi.org/10.1016/j.future.2018.05.044> (2018).
56. Prountzos, A. & Petrakis, E. G. M. DeFog: Dynamic micro-service placement in hybrid cloud-fog-edge infrastructures. *Int. J. Web Grid Serv.* **20**(3), 266–291. <https://doi.org/10.1504/IJWGS.2024.139802> (2024).
57. Microsoft, Windows virtual machines pricing (2025). <https://azure.microsoft.com/en-us/pricing/details/virtual-machines/windows/#pricing>

## Author contributions

A.K. contributed to the conceptualization, methodology, software development, investigation, and drafting of the original manuscript. F.U. oversaw conceptualization, methodology, supervision, project administration, and manuscript review and editing. D.S. was responsible for validation, data curation, formal analysis, and visualization. M.H.K. contributed to conceptualization, formal analysis, and manuscript review and editing. S.A. participated in validation, visualization, and manuscript review and editing. M.T. performed validation, formal analysis, visualization, and manuscript review and editing. All authors reviewed and approved the final manuscript.

## Funding

This research received no external funding.

## Declarations

## Competing interests

The authors declare no competing interests.

## Additional information

**Correspondence** and requests for materials should be addressed to D.S. or M.T.

**Reprints and permissions information** is available at [www.nature.com/reprints](http://www.nature.com/reprints).

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025